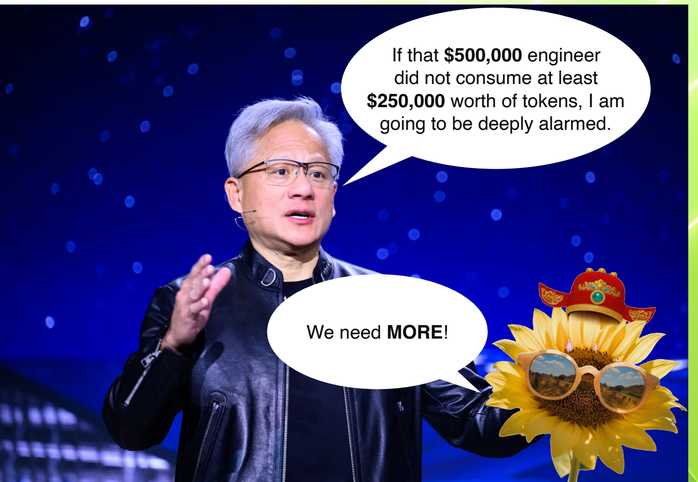
Advancing Token Productivity with Agent Loops

How to effectively transform tokens into productivity.

Ligeng Zhu

NVIDIA Research



2026-Aug: Ligeng's Token Cost

Tokens used
1,590,841,944,468

Lines of code
16,426,139

Monthly usage limit 0% remaining
15,000,006 of 15,000,000 credits used Resets 8:00 AM

2026-July: Ligeng's Token Cost

Tokens used
1,210,508,599,012

Lines of code
8,294,718

Monthly usage limit 26% remaining
5,920,677 of 8,000,000 credits used Resets Aug 1, 2026 8:00 AM

About Me

Scalable and Efficient Agents / Multi-Modal VLMs

Ligeng Zhu

Researcher, NVIDIA Research

Supervised by Prof. Song Han

Research focus

- Agents for L2~L4 level Automatic Research & Engineering
- Scalable and Efficient Agent Harness and Swarms
- Efficient Multi-Modal VLMs

Core Author Projects at NVR

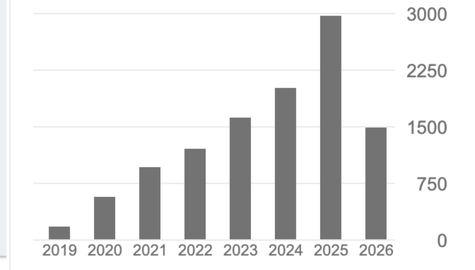
1. **KDA: Kernel Develop Agents for BlackWell**
2. **Humanize 1.0: From Automated Idea Factory to Realization**
3. **JetAutoResearch: A token-efficient harness for agent-driven research**
4. Parason: Revealing Subtask- and Trial Parallelism in LLM Reasoning
5. OckBench: Measuring the Efficiency of LLM Reasoning
6. EGM: Efficient Visual Grounding Language Models
7. Fast-dVLM: Efficient Block-Diffusion VLM via Direct Conversion from Autoregressive VLM
8. NVILA: Efficient Frontier Visual Language Models

Core Author Projects at MIT

1. PockEngine: Sparse and Efficient Fine-tuning in a Pocket
2. On-Device Training Under 256KB Memory
3. Distributed Training Across the World
4. **Deep Leakage from Gradients (most cited paper at MIT HAN Lab)**
5. ProxylessNAS

- Achieved 10k cites this year

Citations	11057	10267
h-index	27	26
i10-index	29	29



Token Maxing with Humanize

O(tokens) = O(productivity)!

- Humanize Ranks #1 on Lean-Eval Leaderboard, Surpassing 2nd Harmonic ($2B valuation) by 30 Questions.
- Humanize Produces >20 Top-5% Solutions on Kaggle, FSD by Agents.
- KDA-v0.5 Optimizes MSA Indexer Kernel by 3.3x in Productions.
- Humanize2 Accelerates Quantum Design by Improving the 5.1% pf global optimum.
- KDA-v0.5 Helps SGLang-Diffusion to Ship Faster and Visually-Lossless Kernels.
- KDA-v0.5 Surpassed all human SOTAs in MLSys FlashInfer Contest.
- Humanize to optimize SASS SOL to 94.9% on Feynman FMHA Backward
- Humanize Tops Quantum Benchmarks QAlg (100%) and QIT (92.5%).
- Humanize Achieves Perfect Scores on IOI, IPhO, ICho, IBO 2026.
- Humanize ACED 6/6 IMO2026 with GPT-5.6 and Kimi-K3
- KDA-v0.1 to Optimize 28.1% mHC for DeepseekV4-Pro (Merged into TRTLLM)
- Humanize Yields Goal-Medal Scores on Physics Cup / SuperChem
- KDA-v0.1 Boosts Databricks to Rank #1 the SOLExec-Bench L1 Single Operations
- KDA-v0.1 Helps Fixing Numerical Issues on SGLang-Omni
- Humanize Achieves 99.7% (670/672) on PutnamBench (from 1963-2023)
- KDA-v0.1 Boosts SGLang Development, optimizing >40 kernels in a day
- KDA-v0.1 Achieves TOP3 on All tracks at MLSys FlashInfer Contest

2026-Aug

Tokens used	Lines of code
1,590,841,944,468	16,426,139

2026-July

Tokens used	Lines of code
1,210,508,599,012	8,294,718

..... more interesting stuffs happening at NVIDIA Internal

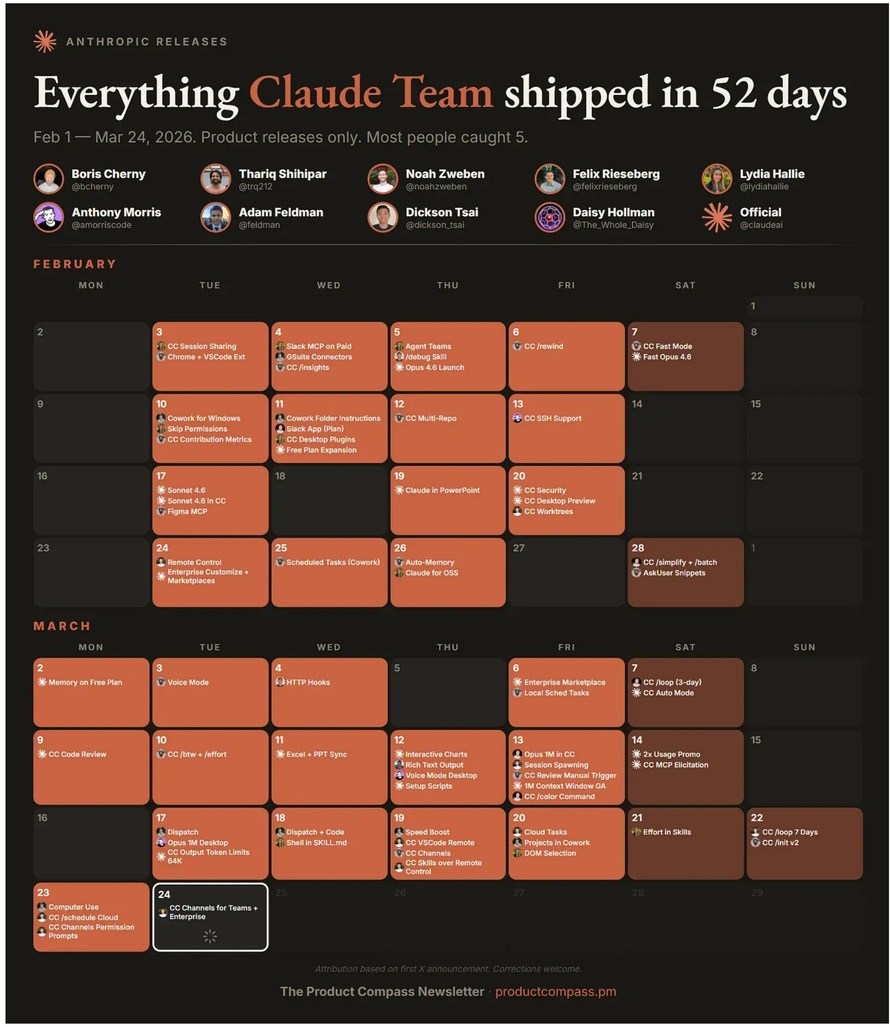
All done in this summer (June to August, ~2 months), w/ a small team (~10 members)

Tokens are **revolutionizing** the industry and academia.

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

AI Coding Evolves So Quickly

- The Power of Scaling
  - **More Knowledge** — the details/grammar/syntax/document of common software
  - **Longer Context** — memorize the whole conversation and not miss any clues for bugs
  - **Better Instruction Following** — knows what to do and what not to do
  - **More Accurate Function Calling** — from "assistants" (tools that suggest) to "agents" (tools that act)
  - **More Intelligent Reasoning** — decompose a complex task into multiple simple ones before execution
  - ……
- In August 2026, we are at the dawn of **Software AGI**



The Progress of AI Coding

2020

CodeBERT Microsoft 2020-01

2021

GitHub Copilot Microsoft 2021-06
OpenAI Codex OpenAI 2021-07

2022

Amazon CodeWhisperer AWS 2022-06
ChatGPT (GPT-3.5) OpenAI 2022-11

2023

Cody, Sourcegraph 2023-06
DeepSeek Coder V1, 2023-11
JetBrains AI, JetBrains, 2023-12
Cursor, AnySphere, 2023-12

2024

Devin, Cognition Labs, 2024-03
Claude 3 Family, Anthropic, 2024-03
CodeGemma, Google, 2024-04
Cursor (Agent Mode). Anysphere, 2024-06
Cline (from Claude Dev), Cline, 2024-10

2025

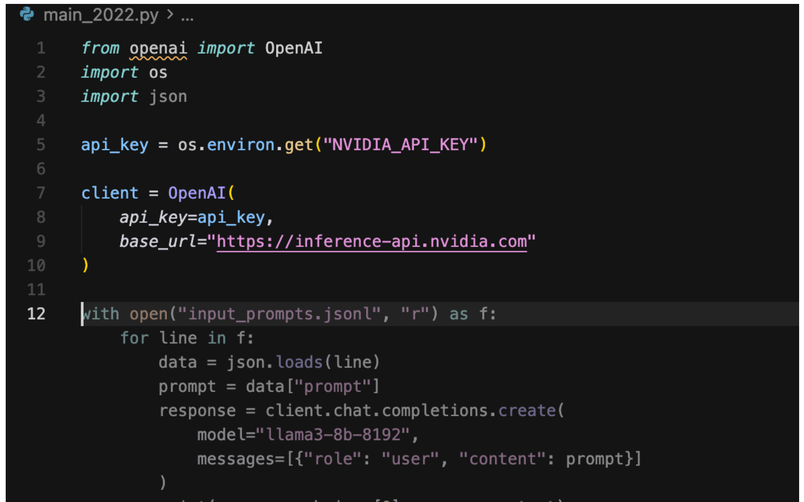
DeepSeek-R1, DeepSeek, 2025-01
Claude Code (CLI), Anthropic, 2025-02
OpenAI Codex (CLI), OpenAI, 2025-05
Claude 4 Sonnet/Opus, Anthropic, 2025-05
Cursor 1.0 (Public GA), Anysphere, 2025-06
Cursor 2.0 (Public GA), Anysphere, 2025-10
Kimi Cli, Moonshot AI, 2025-10
Antigravity, Google, 2025-11
OpenClaw, Personal, 2025-11

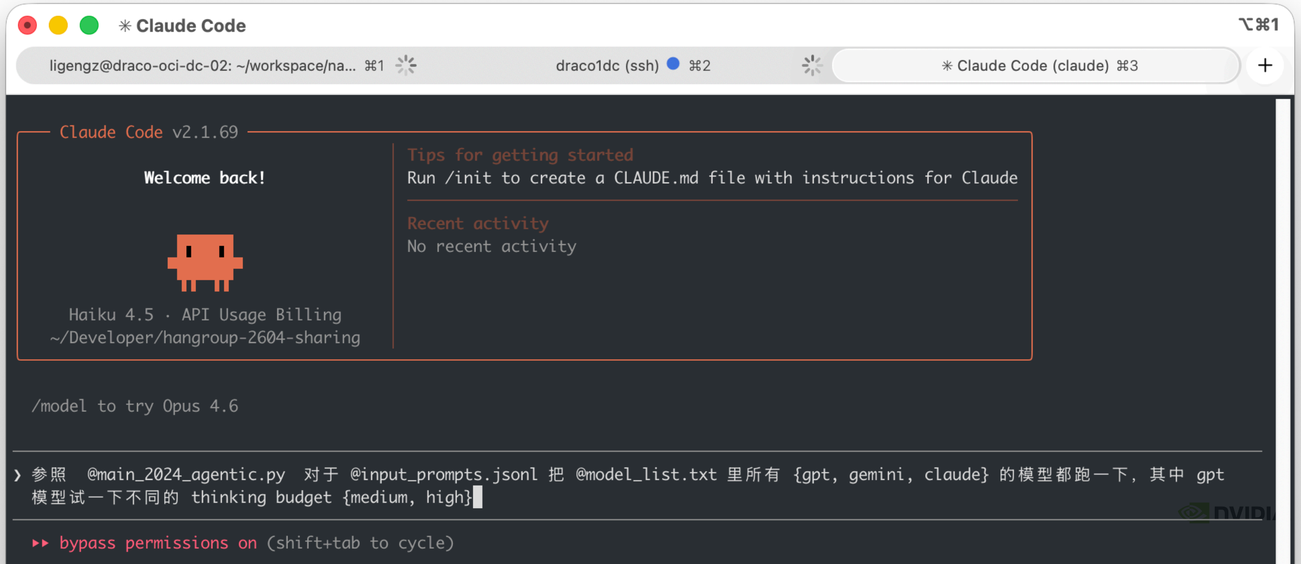
2026

Claude 4.6 Opus, Anthropic, 2026-02
Codex App, OpenAI, 2026-02
GPT-5.3-Codex, OpenAI, 2026-02
Claude Code 2.0, Anthropic, 2026-03
GPT-5.6 Sol, OpenAI, 2026-06
Claude 5 Fable, Anthropic, 2026-07
Claude 5 Opus, Anthropic, 2026-07
Kimi K3, Anthropic, 2026-07
GPT-6 Astra, OpenAI, 2026-(?)
… more to come

Milestones of AI Coding

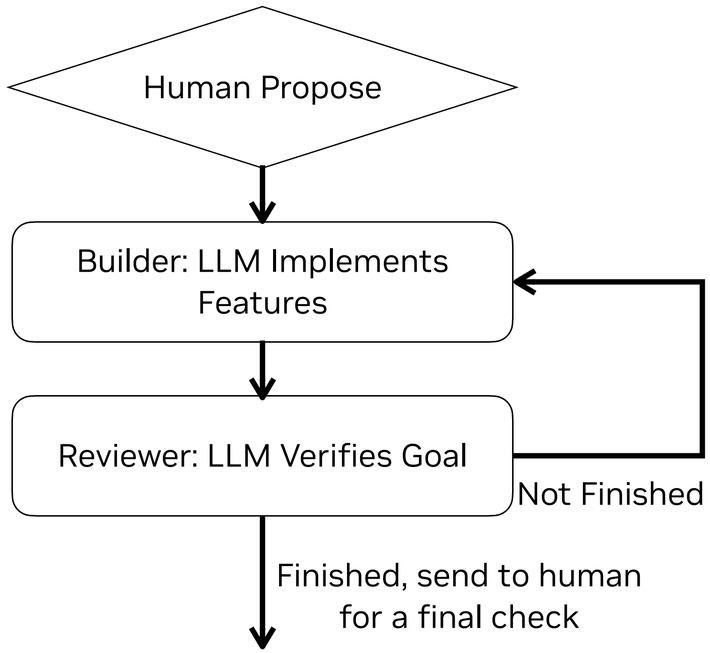
- 2020–2022 (**Autocomplete Era**): Focused on "Ghost Text"—finishing the line of code you were currently typing.
- 2023 (**Contextual Integration**): Focused on "RAG" (Retrieval-Augmented Generation)—understanding your whole folder, not just one file.
- 2024 (**The Agentic Shift**): from "assistants" (tools that suggest) to "agents" (tools that act). Focused on "Action"—running tests, fixing bugs autonomously in the terminal, and managing deployments.
- 2025 (**Reasoning, Multi-Step Agents**): The release of DeepSeek-R1 in January 2025 proved that reinforcement learning could enable open-weight models to compete with proprietary systems in complex multi-step logic.
- Mid-2025 to Early-2026 (**Autonomous Operations / Long Horizon Agents**): Tools that handle the entire engineering workflow from a ticket to merged pull request. Multiple parallel agents cooperate as a team/group/company, assign/delegate/verify the tasks asynchronously and in parallel.





Agent Loops for Autonomous Operations

- Previous **vibe coding**: **single** agent session, **many** prompts
- Current **auto coding**: **many** agent sessions, **singe/few** prompts
- **Core**: Loop with Agents/LLMs!
  - GUI -> CLI, chain agents inputs and outputs, then design your own workflow.



1. Flexibly change the Builder and Reviewer (Opus, GPT, Gemini, Codex, Kimi).
2. Freely change the workflow as you likes, for example
    1. Implement different parts w/ different LLMs (Gemini for frontend, GPT for backend, Claude for networking)
    2. Verification by multiple rounds / voting (accepted only when Gemini + GPT + Opus all agrees).

Such flexibility is not gonna achieved by GUIs.

Agent Flow ***minimizes the human efforts*** .

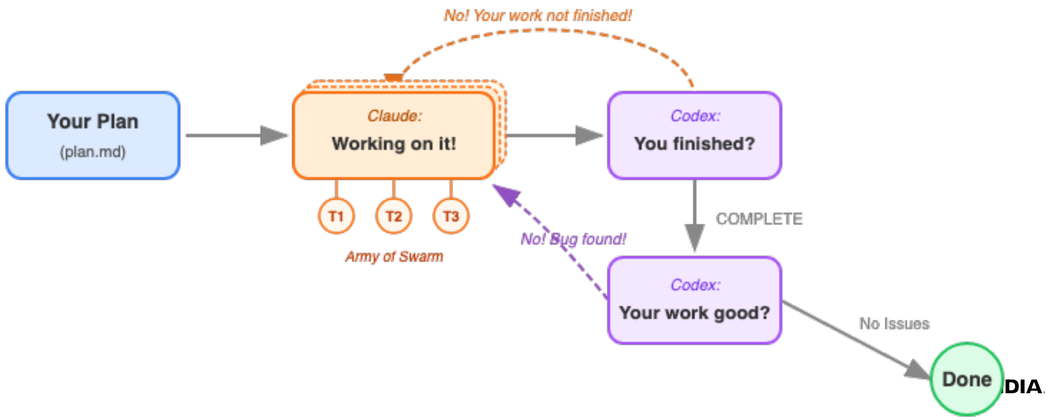
Advancing Modern Agent Loops

- Agent Loop is bit like real-world RL (input from human, mid reward from llm, and final judge by human)
  - Reinforcement Learning with Code Review

- https://github.com/humania-org/humanize
  - A side project from Sihao Liu (UCLA PhD -> NVR), initiated on Jan 2026
  - Deep Collaboration w/ Ligeng Zhu

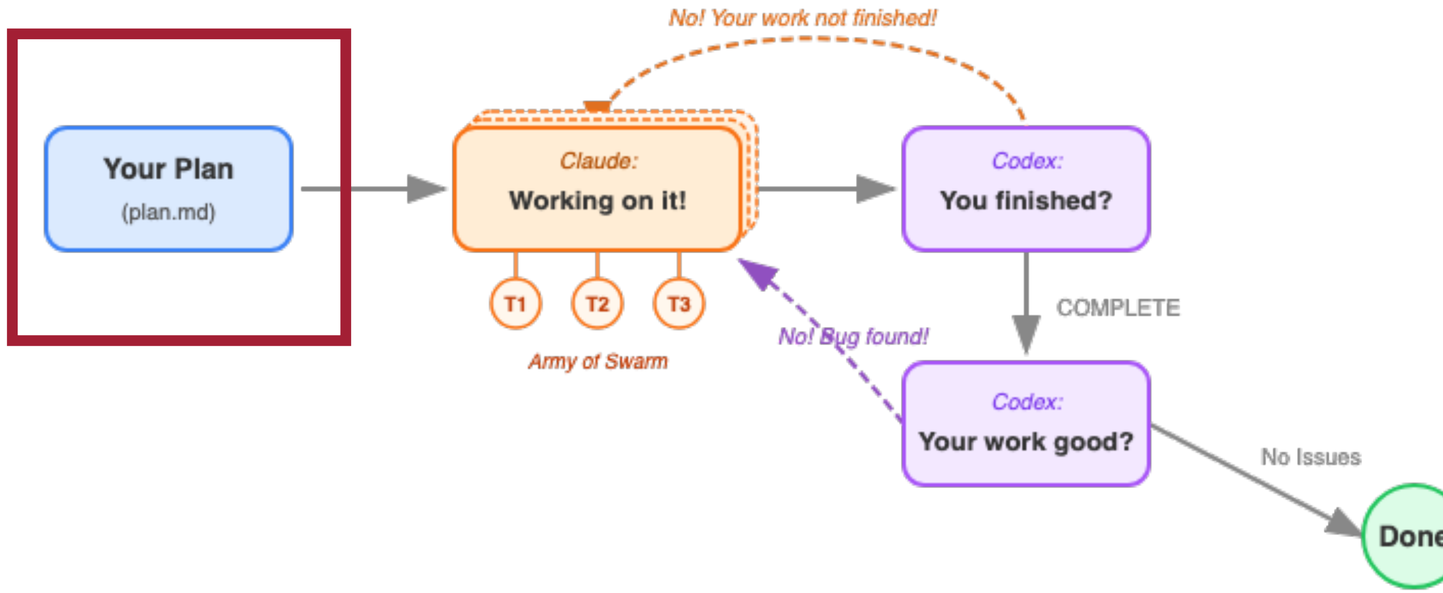
- Core Loops:
  - **Iteration over Perfection** -- Instead of expecting perfect output in one shot, Humanize leverages continuous feedback loops where issues are caught early and refined incrementally.
  - **One Build + One Review** -- Claude implements, Codex independently reviews. No blind spots.
  - **Begin with the End in Mind** -- Before the loop starts, Humanize verifies that you understand the plan you are about to execute. The human must remain the architect.

- Humanize is a pro-max-ultra version of AutoResearch.



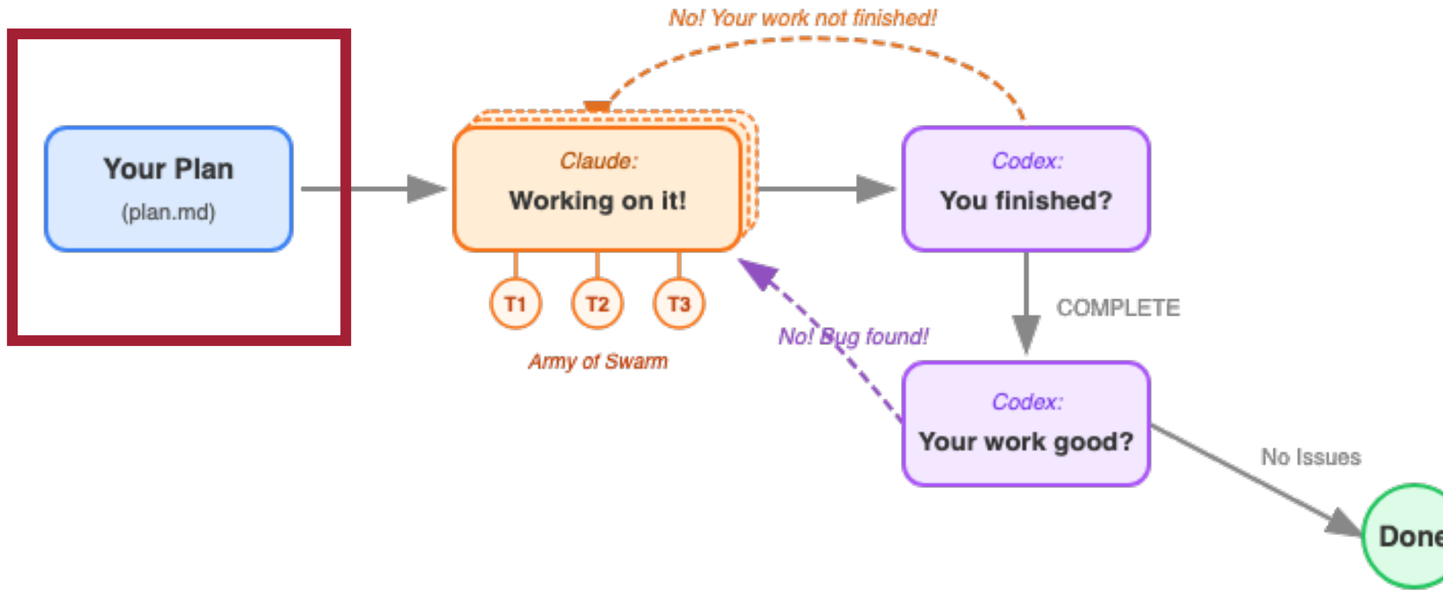
Advancing Modern Agent Loops

- git clone https://github.com/fla-org/flash-linear-attention



Advancing Modern Agent Loops

- git clone https://github.com/fla-org/flash-linear-attention
- vim draft.md
- **> read https://arxiv.org/pdf/2510.26692, based the repo, implement Kimi Delta Attention in CUDA**




```
cat draft.md
```

- `read https://arxiv.org/pdf/2510.26692, based the repo, implement Kimi Delta Attention in cuda`

Advancing Modern Agent Loops

- git clone https://github.com/fla-org/flash-linear-attention
- vim draft.md
- > read https://arxiv.org/pdf/2510.26692, based the repo, implement Kimi Delta Attention in cuda
- **› /humanize:gen-plan --input draft.md --output docs/plan.md**

✻ Running… (4m 25s · ↓ 13.3k tokens · thought for 10s)

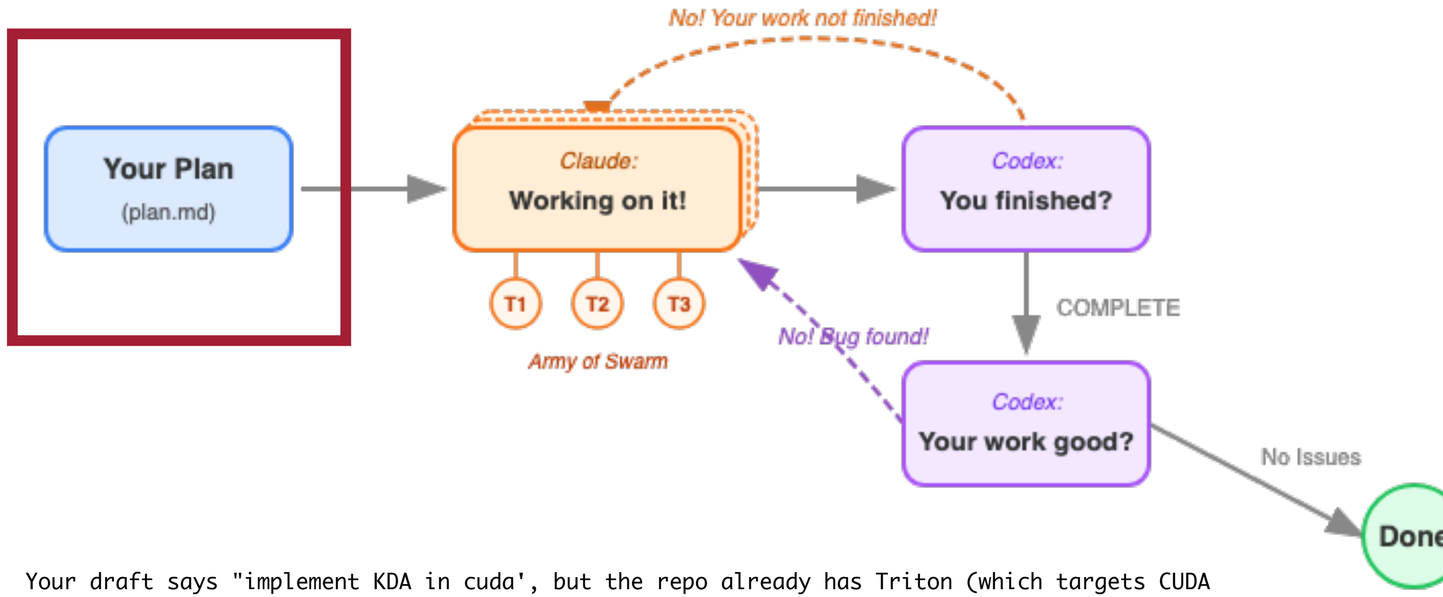
⎿ □ Phase 2: Relevance check via agent

□ Phase 3: Codex first-pass analysis

□ Phase 4: Claude candidate plan v1

□ Phase 5: Iterative convergence loop with second

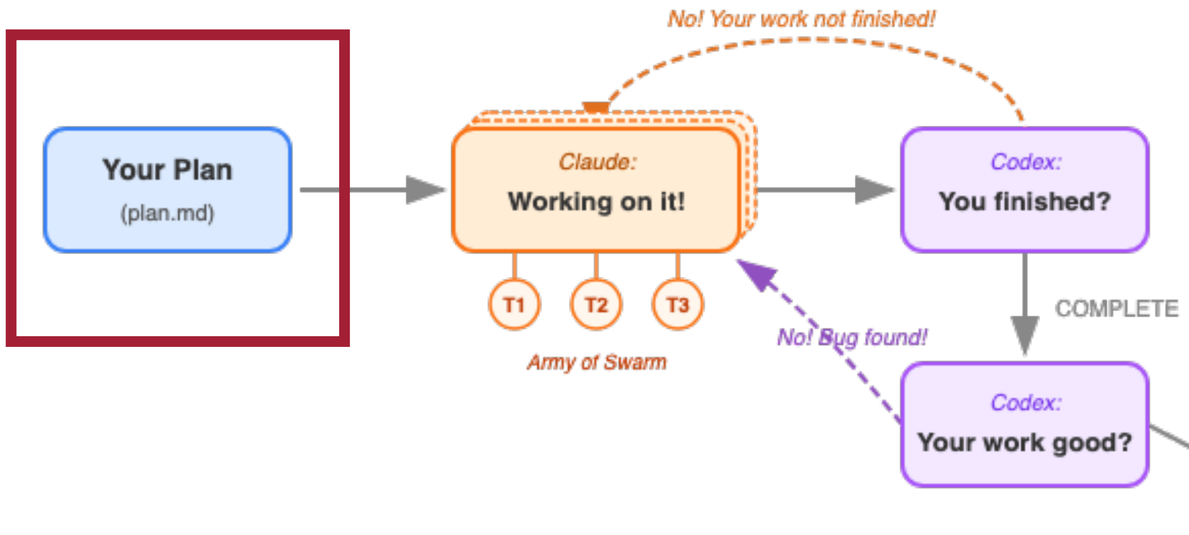
□ Phase 6: Issue and disagreement resolution




```
Your draft says "implement KDA in cuda', but the repo already has Triton (which targets CUDA
GPUS) and an external CUTLASS 'FlashKDA' backend that's inference-only with bf16 + K=V=128 + no
GVA. What scope should the new CUDA backend cover?
› 1. Training (fwd + bwd) New in-tree CUDA C++ / CUTLASS backend with full backward pass (grads
for q, k, v, g, beta, A_log, dt_bias). This is the differential value vs existing inference-
only FlashKDA. Largest effort but unlocks training speedups.
> 2. Inference only. Forward-only CUDA backend; broader inference coverage than FlashKDA.
Smaller effort but overlaps with existing FlashKDA on its shapes; differential is breadth
(dtypes, head dims, GVA, etc.).
> 3. Replace Triton entirely Remove Triton implementation and use only CUDA. Highest risk;
likely loses GVA/varlen/CP coverage. NOT recommended.
4. Type something.
```

Advancing Modern Agent Loops

- git clone https://github.com/fla-org/flash-linear-attention
- vim draft.md
- > read https://arxiv.org/pdf/2510.26692, based the repo, implement Kimi Delta Attention in cuda
- ❯ /humanize:gen-plan --input draft.md --output docs/plan.md
- **vim draft.md # add <cmt> to comment**
- **/humanize:refine-plan --input docs/plan.md**



The implementation is additive: the existing Triton kernels in `fla/ops/kda/` and the existing external `FlashKDABackend` (inference-only) remain in place and continue to win backend selection on their respective supported shapes.

<cmt> Support NVFP4 on BlackWell as well </cmt>

v1 Support Matrix (explicit)

- dtype: bf16
- head dims: K = V = 128
- path: chunk only (no `fused_recurrent` CUDA path in v1)
- chunk_size ∈ {32, 64}
- safe_gate = True with lower_bound in [-5, 0)
- state_v_first = True
- features supported in-kernel: L2-norm fusion, beta-sigmoid fusion, per-key-dim gate fusion, `initial_state` and `output_final_state` (both True and False)
- training: full backward gradients for q, k, v, g, beta, A_log, dt_bias
- features explicitly rejected at predicate (registry falls through to Triton): GVA (HV != H), varlen / `cu_seqlens`, CP-active state, `state_v_first=False`, `allow_neg_eigval=True`, `return_intermediate_states=True`, K != 128 or V != 128, `safe_gate=False`, dtype != bf16, `initial_state.requires_grad=True`, `torch.is_grad_enabled() == False`, no tensor with `requires_grad=True`

Acceptance Criteria

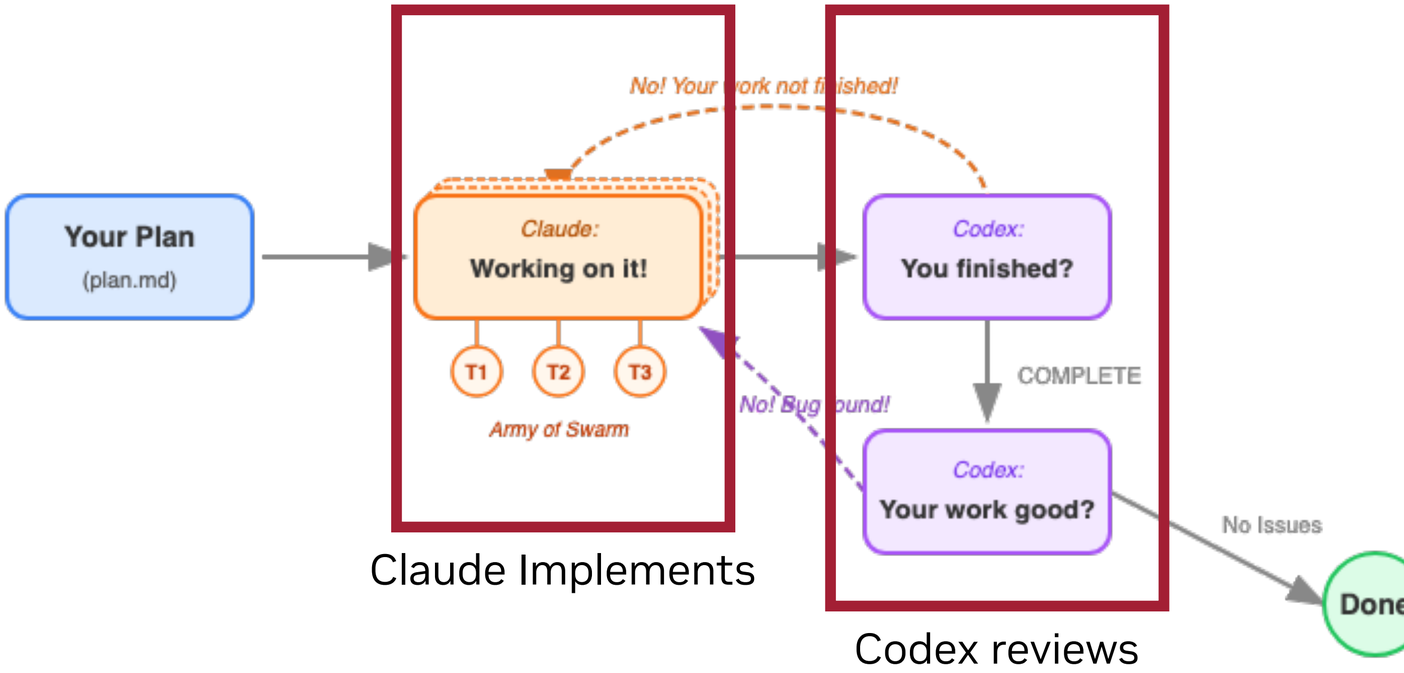
Following TDD philosophy, each criterion includes positive and negative tests for deterministic verification.

- AC-1: Forward numerical parity with reference implementations on the v1 matrix.
  - Positive Tests (expected to PASS): **<cmt> rtol < 1e-4 </cmt>**
    - Output of the new CUDA backend matches `naive_chunk_kda` within `max-abs ≤ 5e-3` and `rms ≤ 1e-3` across (B ∈ {1, 2, 4}, T ∈ {512, 2048, 8192}, H ∈ {4, 8}, K = V = 128, chunk_size ∈ {32, 64}, safe_gate=True, all in-kernel fusions ON, both with and without `initial_state`).

NVIDIA

Advancing Modern Agent Loops

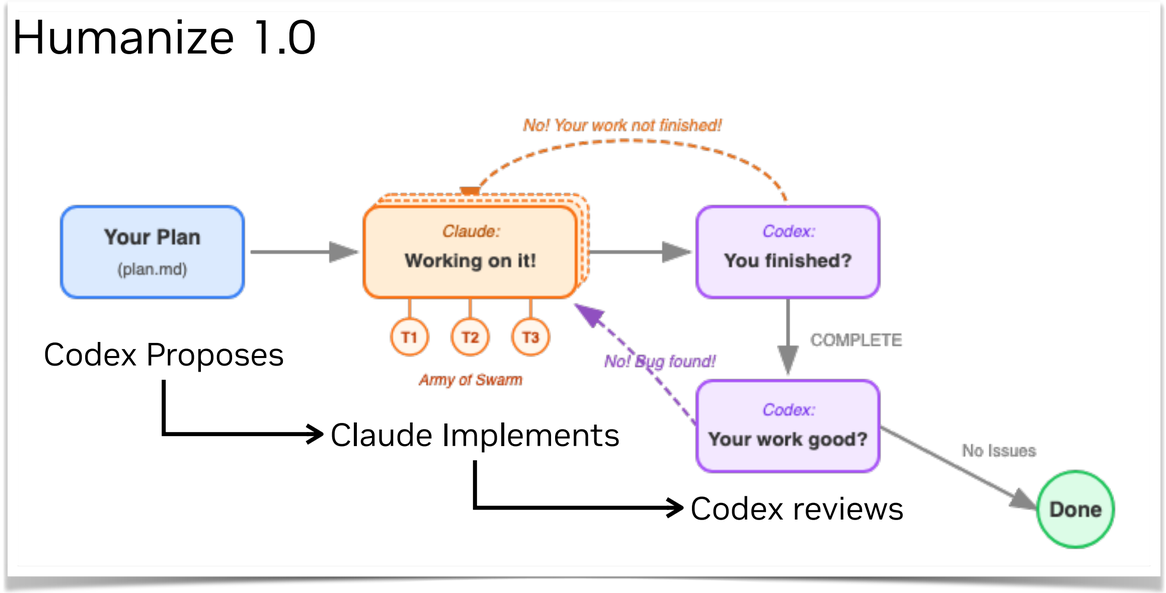
- git clone https://github.com/fla-org/flash-linear-attention
- vim draft.md
- > read https://arxiv.org/pdf/2510.26692, based the repo, implement Kimi Delta Attention in cuda
- ❯ /humanize:gen-plan --input draft.md --output docs/plan.md
- vim draft.md # add <cmt> to comment
- /humanize:refine-plan --input docs/plan.md
- **/humanize:start-rlcr-loop docs/plan.md**



Humanize: Advancing Modern Agent Loops with Flow-Level Designs

```
Vibe Code / Vibe Research =
   Model  (opus/gpt/deepseek/kimi) X
   Tool   (claudecode/codex/pi/dsh) X
   Flow   (humanize/autoresearch/ralphaloop)

Agent   = Model X Tool
Harness   = Tool X Flow
```



Model-level
(5~10mins)

```
response = client.responses.create(
  model="gpt-5.5",
  input="Profile FP8MoE and improve the
implementation."
)
# act blabla
```

Tool-level
(30mins ~ hours)

```
 >_ OpenAI Codex
| permissions: YOLO mode

› Profile FP8MoE @fp8_moe.py and improve
the implementation
```

Flow-level
(days ~ weeks)

```
› /humanize:gen-plan --input draft.md --
output docs/plan.md
› /humanize:refine-plan --input docs/
plan.md
› /humanize:start-rlcr-loop docs/plan.md
```

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

Different Level of Vibe Coding

Model-level (1~10mins)

```
response = client.responses.create(
  model="gpt-5.5",
  input=“Profile FP8MoE and improve the implementation."
)
# act blabla
```

Tool-level (10mins ~ hours)

```
 >_ OpenAI Codex
| permissions: YOLO mode

› Profile FP8MoE @fp8_moe.py and improve the implementation
```

Flow-level (days ~ weeks)

```
› /humanize:gen-plan --input draft.md --output docs/plan.md
› /humanize:refine-plan --input docs/plan.md
› /humanize:start-rlcr-loop docs/plan.md
```

Test-Time Scaling!

GPT-5.5	Physics Cup	SuperChem	HLE (Subset)
Model Level (API)	44.0%	47.0%	29%
Tool Level (CLI)	66.0%	56.8%	42%
Flow Level (Humanize)	**80.0%**	**62.4%**	**63%**

KDA: Kernel Design Agents for BlackWell

Automatic Kernel Pipelines

Powered *by Token*, Powered *for Token*

Ligeng Zhu

NVIDIA Research

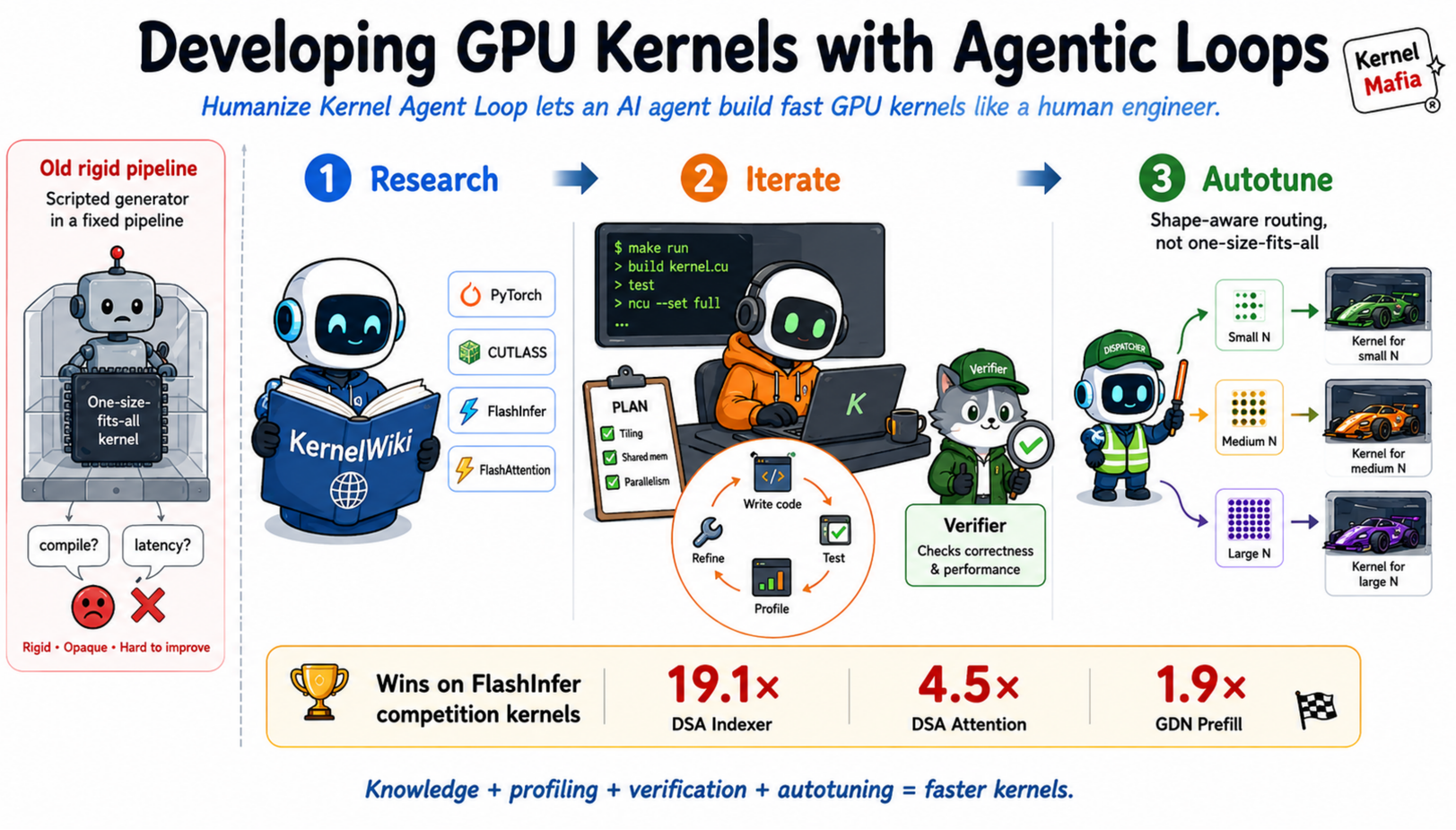
Method overview of KDA

Modern SOTA LLMs are powerful, but to unleash its full power, we need Model X Tool cooperation.

1. **Humanize:** Long running agents loops

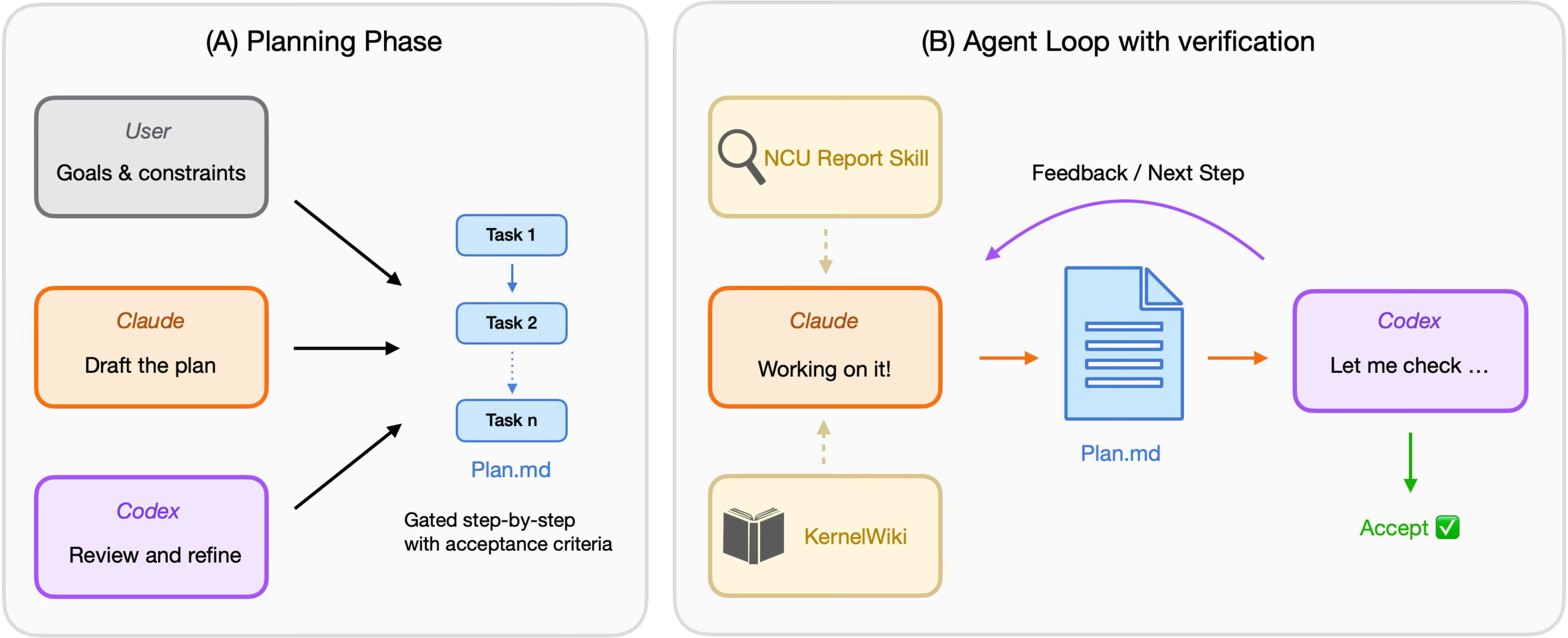
2. **KernelWiki**: A curated implementation reference from SGLang, vLLM, TRTLLM

3. **NCU-Profile.skill**: analyze the detailed roofline and performance trace.



Humanize Agentic Loop

Coder and verifier work inside the real development environment

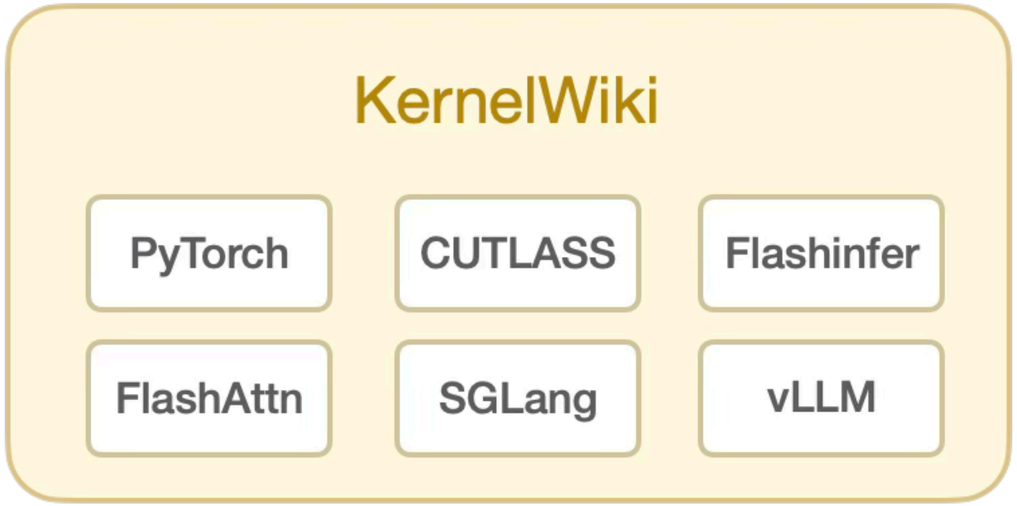


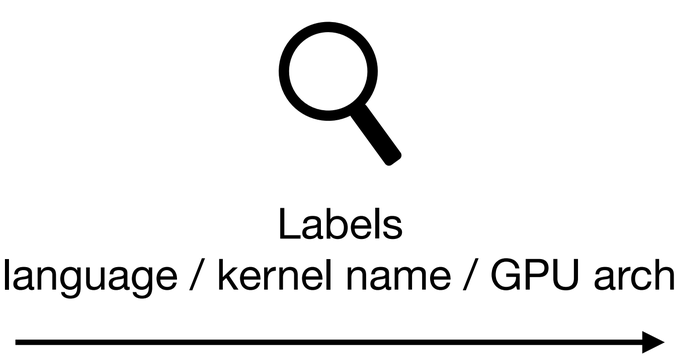
https://github.com/PolyArch/humanize

NVIDIA

Kernel Wiki

Domain knowledge as retrievable agent memory





Reference

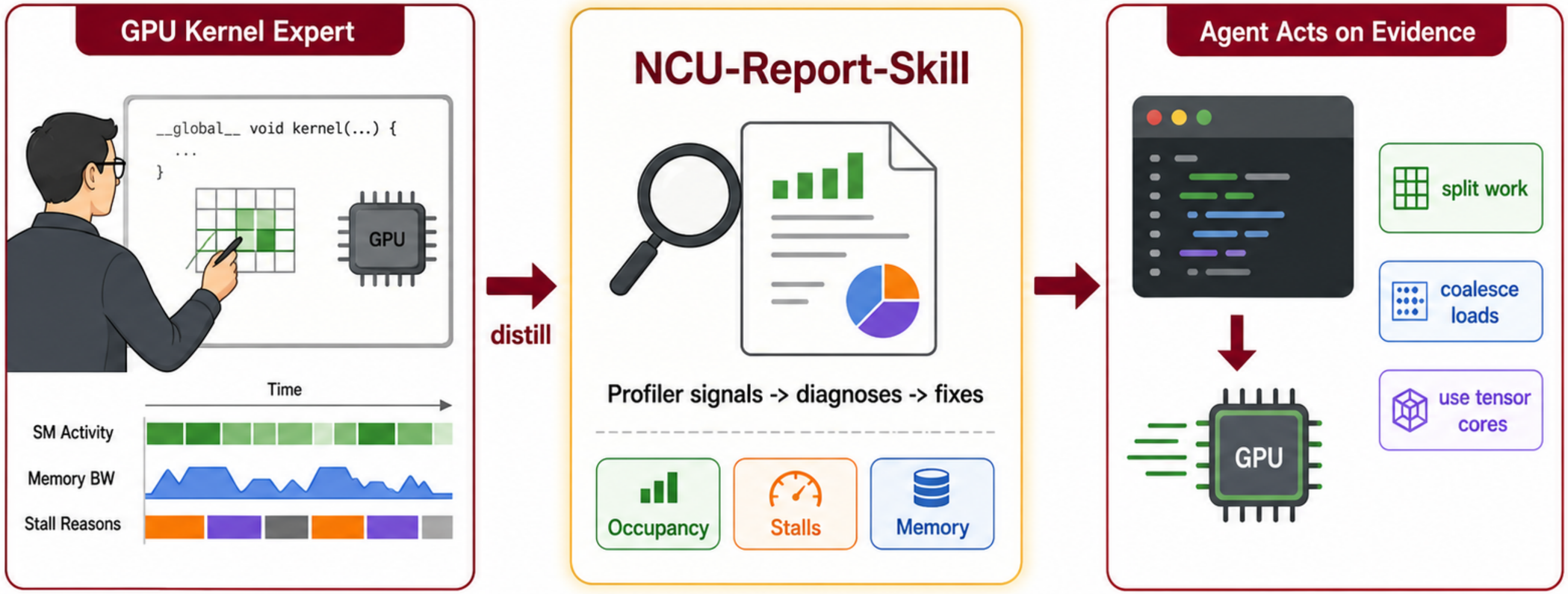
1. Collect PRs from famous repo
2. Filter by keywords: Blackwell/sm100/moe …
3. Use Claude to generate label for each PR

SGLang/vLLM/
TensorRT-LLM/PyTorch

NVIDIA

NCU-Report-Skill

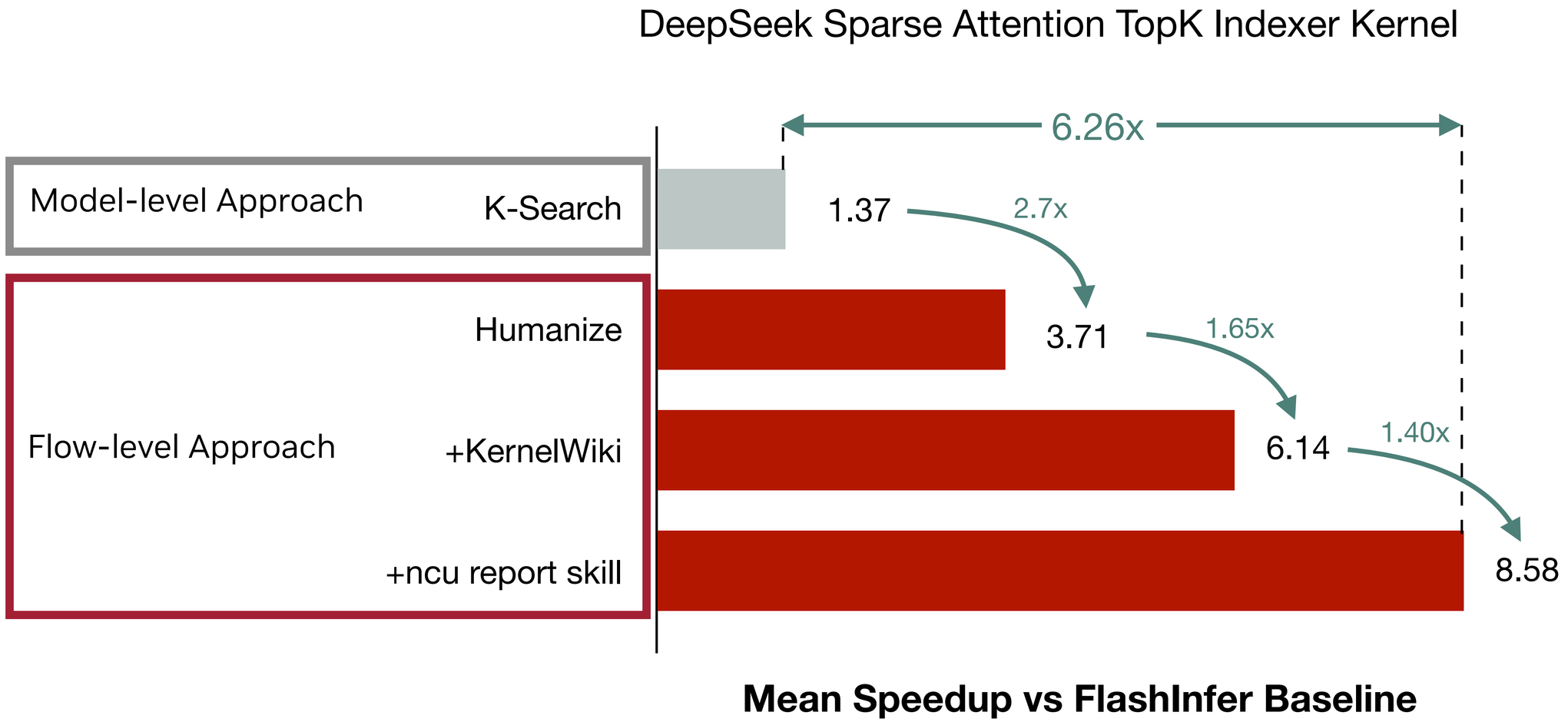
Distill Expert Profiling Workflow into an Agent Skill



Expert knowledge + profiler evidence = actionable optimization

KDA's General Improvement

Agentic loop w/ little human priors brings the impressive improvements.



KDA's Detailed Analysis on DSA Indexer

- **Official Implementation:** First compute the scores using a scalar FP8 score kernel, then run radix-select.
  - Bounded by the score stage: high register pressure, low occupancy, and the FP8 dot product does not go through the Blackwell tensor core path.
- **KDA's Approach**:
  - KernelWiki changed the search direction by bringing Hopper-to-Blackwell migration notes and production-kernel experience from projects like vLLM and DeepGEMM into the loop. Instead of continuing to locally tweak the *scalar dot-product* path, it rewrote score computation into a *Blackwell-style tensor-core* path:
    - `tcgen05.mma` instruction path;
      - **This knowledge was not trained until Opus 4.8 (we use Opus 4.6 during the contest)**
    - TMA-based paged loads;
    - TMEM-backed accumulation;
    - warp-specialized execution.
  - the score stage dropped from ***73.6 μs* to *7.1 μs***, and only then did optimization effort shift to radix-select and workload-specific fast paths.
- KDA tried multi-block radix-select, but validation showed synchronization and launch overhead canceled out the potential gains
  - **but TRTLLM devs said it can actually improve the speedup, agents failed to find.**
- The submitted version ultimately achieved **19.1× mean** speedup.

KDA's Detailed Analysis on DSA Sparse Attention

- The core challenge of Sparse Attention is to efficiently **gather**, **filter**, and **reduce** over the selected KV subset, while also mitigating the non-contiguous memory access issues introduced by sparsity.

- **KDA's first version achieved stable gains mainly via a split-KV design:**
  - 16-way parallelism, ballot-and-popcount sparse filtering, vectorized shared-memory gather, and log-sum-exp reduction. This path exploits real sparsity and reduces the wasted computation present in the official baseline.
- KDA then continued to explore:
  - single-CTA fusion
  - cooperative split-reduce fusion
  - cluster-based launch patterns, etc.
  - But under verifier-grounded measurement, many branches were neutral or even negative.
  - Finally, a more suitable eager kernel family and structural fusion that reduces launch overhead.

- In the end, the submitted DSA Attention version achieved a **4.5× mean speedup** over the official baseline and passed **all 23 official workloads**.

KDA's Detailed Analysis on GDN Track - Prefill

- GDN Prefill is to build a custom CUDA "short path" for short sequences: keep the state in registers and fuse scale, softplus, sigmoid, gating, and the state update, avoiding the excessive pipeline startup, TMEM lifecycle, scheduler overhead, and epilogue cost that the CuTe baseline pays on short sequences.

- **On short-sequence workloads**, this path delivers clear gains:：
  - seq_len = 6 ~8.05x speedup； seq_len = 30 ~2.0x speedup； short workload average ~3.94x。
- **But the conclusions are different for long sequences.** The agent tried roughly twenty approaches (including chunk-wise kernels, medium-length scalar extensions, 2-SM cooperative MMA, and fusion variants), but none passed robust verification. `ncu-report-skill` indicates the main bottleneck for long sequences is long-scoreboard memory latency, not the SFU overhead that might be the initial assumption.

- Therefore, the final solution uses a **hybrid dispatcher**:
  - short sequences take the custom CUDA path,
  - long sequences fall back to the untouched CuTe baseline.

- kernel optimization does not always mean ONE implementation for ALL shapes. For some shapes, it's more reliable to acknowledge that a mature baseline is stronger, and use a router to switch to a custom path only when it's a good fit—often a better engineering choice overall.

- the submitted GDN Prefill version achieves a 1.9× arithmetic-mean speedup.

KDA's Detailed Analysis on MoE Track

- This benchmark comes from DeepSeek-V3/R1's FP8 block-scaled mixture-of-experts kernel, which combines routing, FP8 block dequantization, grouped GEMM scheduling, and weighted accumulation into a single scored operator. **The official baseline calls a closed-source TRT-LLM FP8 GEMM binary**, meaning the visible implementation cannot fully explain where the baseline performance comes from.

- KDA initially tried to reproduce these capabilities in CUDA, **but did not converge within the contest timeframe**. The direction later shifted to Triton, with an explicit requirement that the agent use Triton 3.6's Blackwell support: check whether the generated PTX hits the native tcgen05 FP8 path, and then test hardware-scaled FP8 GEMM variants.

- In the end, the submission ranked **1st place in the MoE Track**; however, compared to the official baseline, the submission's mean speedup was **0.65x**, i.e., it did not surpass the baseline. More precisely, the system found some effective directions, but it still did not fully reproduce the FP8 GEMM capabilities behind the closed-source baseline, and there **remains a gap to a human-expert-level** implementation.

- It's also worth mentioning that after the contest, when there was more time, we had KDA implement an FP8 MoE kernel in CUDA that was **faster than the baseline (1.44x, vs human best 1.68x)**
  - W/ Humanize 2.0 (better Harness design)
  - Longer Running Time
  - The speedup is still improving

KDA: Kernel Design Agents

Modern SOTA LLMs are powerful, but to unleash its full power, we need Model X Tool cooperation.

```
Vibe Code / Vibe Research =
    Model (opus/gpt/deepseek/kimi) X
    Tool  (claudecode/codex/opencode) X
    Flow  (humanize/ship/autoresearch/superpower)
```

1. **Humanize:** Long running agents loops
2. **KernelWiki**: A curated reference from SGLang, vLLM, TRTLLM
3. **NCU-Profile.skill**: analyze the detailed roofline and trace.

	K-Search	KDA-v0.1 (contest ver) May 2026	Human SOTA 2026-05	KDA-v0.5 July May
FP8 MoE	0.27x	0.72x (1st place)	1.80x	2.25x
DSA	6.15x	15.22x (2nd place)	22.99x	29.95x
GDN Prefill	1.09x	1.92x (3rd place)	4.42x	6.10x

Flow-level agents greatly boost CUDA agent performance.

KDA evolves very quickly, after **three months**, and now can **outperform human SOTAs** (25~39%)

Notably: The advisor (ligeng zhu) had not written a single line of CUDA in 2026, and the student (dongyun zou) had only written about <1k lines of Triton in 2026.

Databricks is applying KDA to rank #1 (~90 ops) on SOLExec-bench L1.
SGLang adapted KDA to optimize > 100 kernels.
ModelOPT Team: 19.1% faster LoRA SVDQuant flashinfer/3858
Fast Kernel Team: 26.8% faster mHC TensorRT-LLM/16799

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

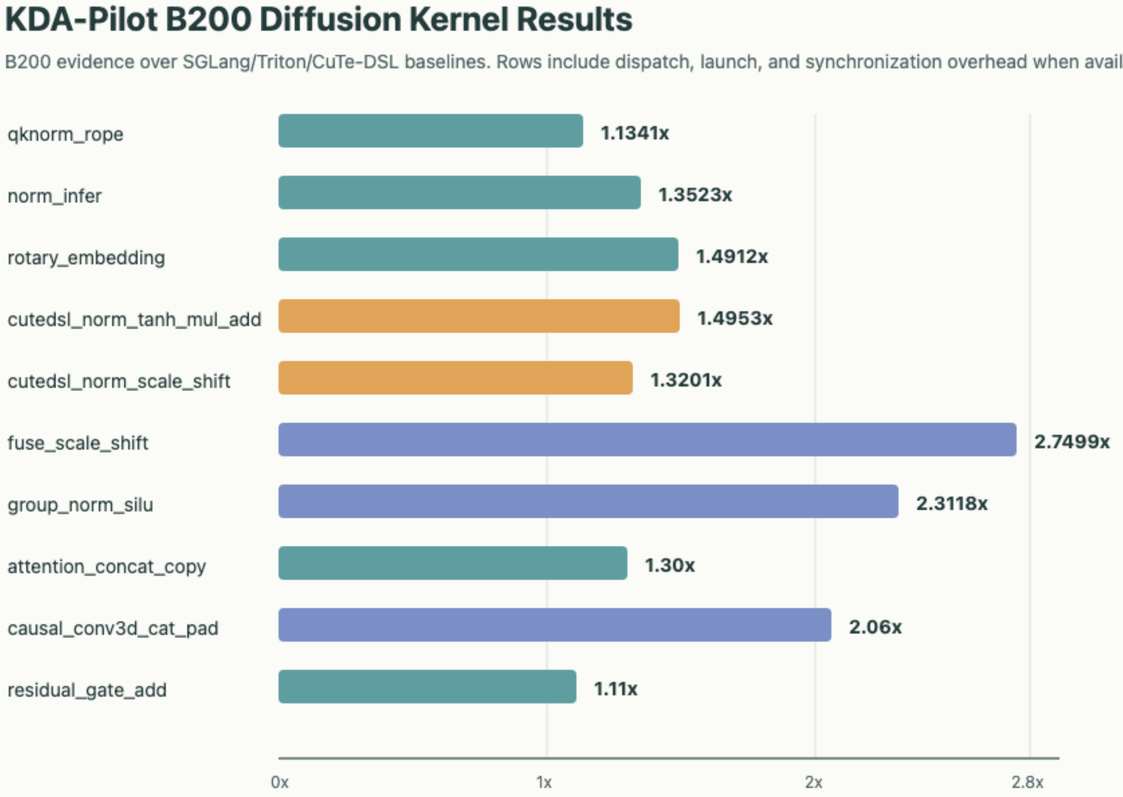
KDA Boosts the SGLang

Performance optimization at SGLang framework (PI: Xiangyu Zhang / Bbuf)

- https://github.com/BBuf/KDA-Pilot (SGLang, RadixArk)
- KDA **completed ~20 of optimizations** for SGLang Diffusion Kernels **in one day**, including 2.7570x geomean for B200 fuse_scale_shift, 2.2794x geomean for B200 group_norm_silu, and 1.27x geomean for H200 norm_scale_shift. The optimized kernels can be used on demand in real model inference frameworks.
- E2E **12% speedup** on well-optimized Qwen-Image.

A lot of improves have been made into PRs

- Router long-context tokenization dedup / PR #28744: idle TTFT 60k -29%, 125k -41%; 60k under load -34% ~ -49%
- Qwen3-Next FlashInfer allreduce fusion / PR #22664: throughput +71.4%; mean TTFT 456.24ms -> 167.54ms, about -63.3%
- Cohere2Moe NVFP4 fused-MoE / PR #27401: throughput chat +26%, summarization +21%; vs. vLLM +4.1% / +6.8%
- Kimi Delta Attention CuteDSL prefill / PR #27488: KDA prefill vs. Triton 1.08x ~ 1.52x
- Spectral Progressive Diffusion / PR #27524: denoising FLUX.1 1.63x, FLUX.2 1.77x, Z-Image 2.07x, Wan 2.32x, Qwen-Image 1.6
- LTX-2 VAE decode channels-last-3d / PR #27431: decode 5.41s -> 3.84s, about 1.41x; latency about -29.0%

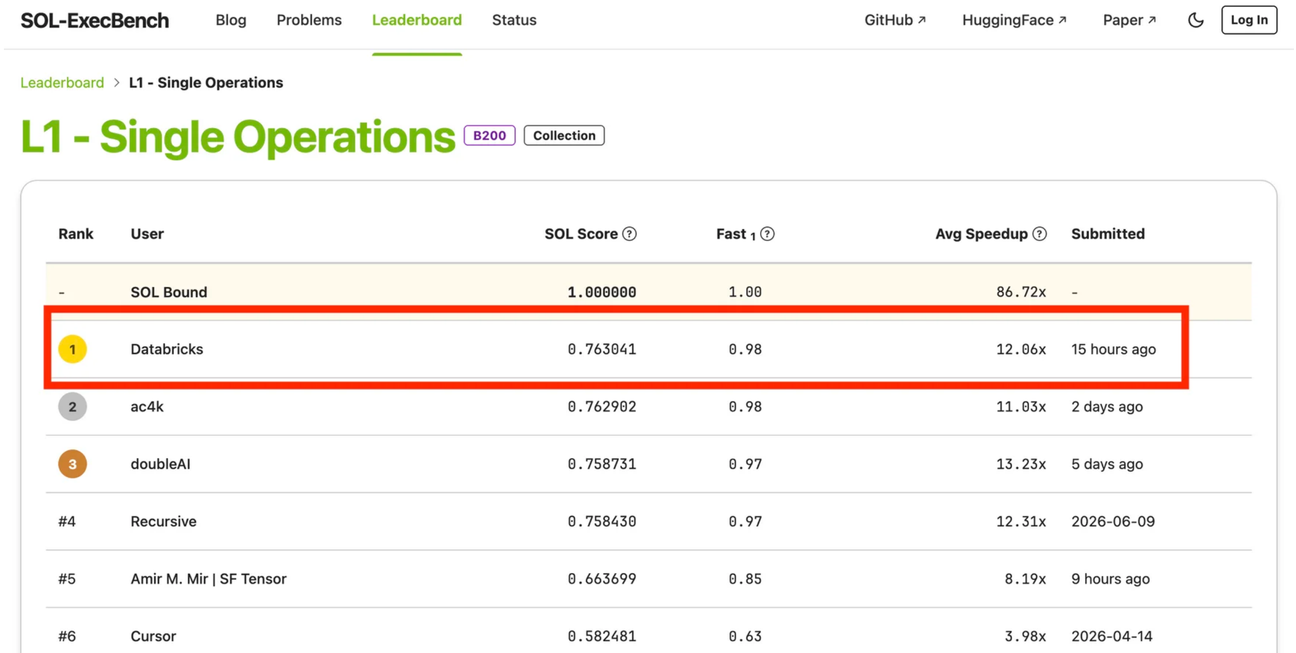


Agent-Assisted SGLang Development: An Initial Exploration

KDA are now generating SOL-Bench SOTA in batches

Databricks is applying KDA to reach top-performance on SOL-bench (PI: Yunchen Jin)

- The SOL-bench L1 single operation track score of **0.7608** has been confirmed as first place; the previous best was **0.7584** by recursive AI (w/o using Fable or GPT-5).
- 1 8xb300 node, $600~900 per OP, within a week
- Let’s, how long will KDA take to dominate L2 / L3 / L4 :)



KDA Accelerated ModelOPT Dev

Diffusion Optimization from ModelOpt Team at NVIDIA (PI: Jingyu Xin)

- KDA successfully reduced SVDQuant kernel latency by up to 30% and quickly implements a **NVFP4 Gemm + BF16 Lora**.
  - 4~12% speedup on Attn and FFN (merging to FlashInfer/TRTLLM)
- KDA Implements a svdquant that is 4% faster than cutlass nvfp4 c++ https://github.com/NVIDIA/TensorRT-LLM/pull/15693 (merged)

KDA Generalizes to More Hardware

AMD Team is also applying KDA on ROCM/FlyDSL to optimize MI355x (PI: Jin Pan)

- ASM, HIP, ROCM: even on less popular less documented hardware (e.g., AMD), our KDA can still find out the optimization opportunity to boost performance (4~6% improvement on FlashAttn)
- KDA Generalizes to More Hardware: FlyDSL/pull/711 FlyDSL/pull/685

This shows the generalization ability of KDA on NVIDIA future Rubin and Groq hardware.

KDA Overall Achievements in Summer

- KDA-0.5 surpassed human SOTA —MoE 2.25×, DSA 29.95×, and GDN 6.10× — **Contest solution repo**.
- SOLExecBench — 1st Place: KDA boosted Databricks to #1 on SOLExec L1, scoring 0.7608 versus the previous best of 0.7584—**Announcement on X.**
- KDA-0.1 achieved TOP3 across every track in the MLSys'26 FlashInfer Contest on B300—MoE 0.67×, DSA 11.91×, and GDN 1.16×
- KDA delivered a 6.1× balanced k-means speedup for the Wan 2.2 workload on B200—**Flash-KMeans #23**.
- Cosmos3 causal Conv3D cat-pad achieved a 2.03× weighted B200 kernel-group speedup, reducing latency from 10.621 to 5.240 ms and improving model E2E performance by 2.11%—**SGLang #29281**.
- LTX-2.3 QKNorm + split-RoPE achieved a 5.84× kernel geomean on B200, with 4.22–7.34× speedups across 14 production shapes and 9.16% lower E2E latency—**SGLang #29708**.
- KDA NVFP4 GEMM for Qwen3.8 on SM120 achieved a 1.21× selected GEMM speedup—**SGLang #36865**.
- Humanize + KDA restored SM121 QSA correctness while delivering 4.45% higher E2E throughput—**SGLang #36845**.
- The Blackwell fused MHC half-MMA kernel achieved a 4.24–19.86% speedup—**TensorRT-LLM #16799**.
- FlashInfer: The SM100 NVFP4 SVDQuant fused GEMM achieved a 1.06× peak GEMM speedup—**FlashInfer #3858**.
- The LTX-2.3 QKNorm + split-RoPE CUDA fast path achieved a 5.84× kernel geomean—**SGLang #29708**.
- The LTX-2 residual-gate CUDA fast path achieved a 1.032× model E2E speedup—**SGLang #29361**.
- The Cosmos3 causal Conv3D cat-pad CUDA fast path achieved a 2.03× weighted kernel-group speedup—**SGLang #29281**.
- Sana packed self- and cross-attention GEMMs achieved a 1.06× denoise speedup—**SGLang #28393**.
- The FLUX fused FeedForward GELU up-projection achieved a 1.015× denoise speedup—**SGLang #28166**.
- Qwen-Image norm-scale-shift achieved a 1.13× denoise speedup—**SGLang #27392**.
- Qwen3-Next FP8 MoE achieved a 1.06× chat-throughput improvement—**SGLang #26204**.

..... more optimizations are on the way!

Agents that Explore Open Science Problem

A shown case of Anthropic's Take-home

Ligeng Zhu

NVIDIA Research

Anthropic's Takehome Challenge

https://github.com/anthropics/original_performance_takehome

- **Goal**: optimizing a batched tree-walk computation on a virtual VLIW + SIMD machine (in two hours)
  - SIMD/SIMT: each cycle, **single** instruction, **multiple** data
  - VLIW: each cycle, **multiple** instruction, **single** data
  - VLIW + SIMD: each cycle, **multiple** instruction, **multiple** data

- Why choose the VLIW + SIMD hardware?
  - VLIW is hard to program to the pipeline — compiler suffers, and takes time to find the best schedule
  - Since the programing difficulty, VLIW + SIMD hardware has not been widely produced / deployed as SIMT/SIMD
    - so little leakage in pretraining or post-training.
    - Domain knowledge is not that helpful

- Therefore, anthropic's blog says such challenges is **AI-resistant**
  - https://www.anthropic.com/engineering/AI-resistant-technical-evaluations

- **Interview Target**:
    - ❌: how the candidate familiar w/ compute architecture/parallel computation
    - ✅: how fast the candidate could design a "harness" to solve the issue

Anthropic's Takehome Challenge

2164 cycles: Claude Opus 4 after many hours in the test-time compute harness
1790 cycles: Claude Opus 4.5 in a casual Claude Code session, approximately matching the best human performance in 2 hours
1579 cycles: Claude Opus 4.5 after 2 hours in our test-time compute harness
1548 cycles: Claude Sonnet 4.5 after many more than 2 hours of test-time compute
1487 cycles: Claude Opus 4.5 after 11.5 hours in the harness
1363 cycles: Claude Opus 4.5 in an improved test time compute harness after many hours

If you optimize below 1487 cycles, beating Claude's best performance at launch, email us at performance-recruiting@anthropic.com with your code and a resume.

Model-level Agents:

9812 cycles: DeepSeek-V3.2
2901 cycles: Opus 4.6
2784 cycles: Kimi-2.6
2544 cycles: Minimax-2.5
2401 cycles: DeepSeek-V4-Pro

Tool-Level Agents:

1864 cycles: Opus 4.6 oneshot
1755 cycles: Opus 4.7 oneshot
1711 cycles: GPT-5.3-codex

——————— (after April 2026) ———————

Simple Flow-Level Agents:

1458 cycles: Claude Code /goal + Opus 4.7
1311 cycles: Codex /goal + GPT-5.5

Flow-Level Agents:

1250 cycles: Humanize 1.0 + GPT-5.4 + Opus 4.6
1001 cycles: Humanize 1.5 + GPT-5.5 + Opus 4.7
994 cycles: Humanize 2.0 + GPT-5.5 + Opus 4.7
979 cycles: Humanize 2.5 + GPT-5.6 + Fable

- Jian Weng (KAUST): 1250 CYCLES
- NVR US: Bian Song 1007 cycles
- NVR: **Zijian Zhang & Ligeng Zhu** 994 cycles (941 w/ some tricks)
- NVR: **Xingyang Li** 979 cycles
- NVR: **Alan & Ligeng Zhu** 965 cycles

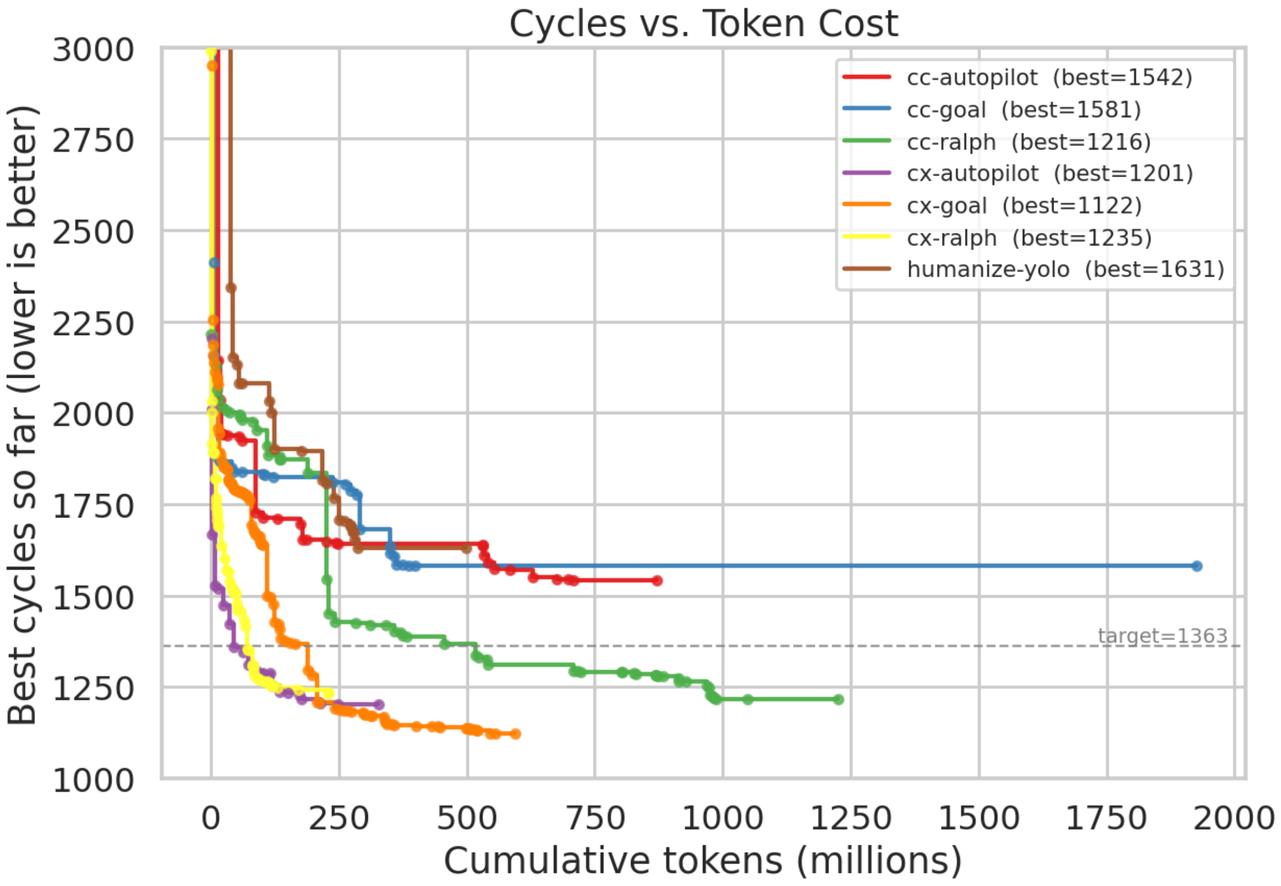
Theoretical limit (read all data) is around ~930 cycles.
965 cycles << 1,487 cycles (a\'s interview bar)

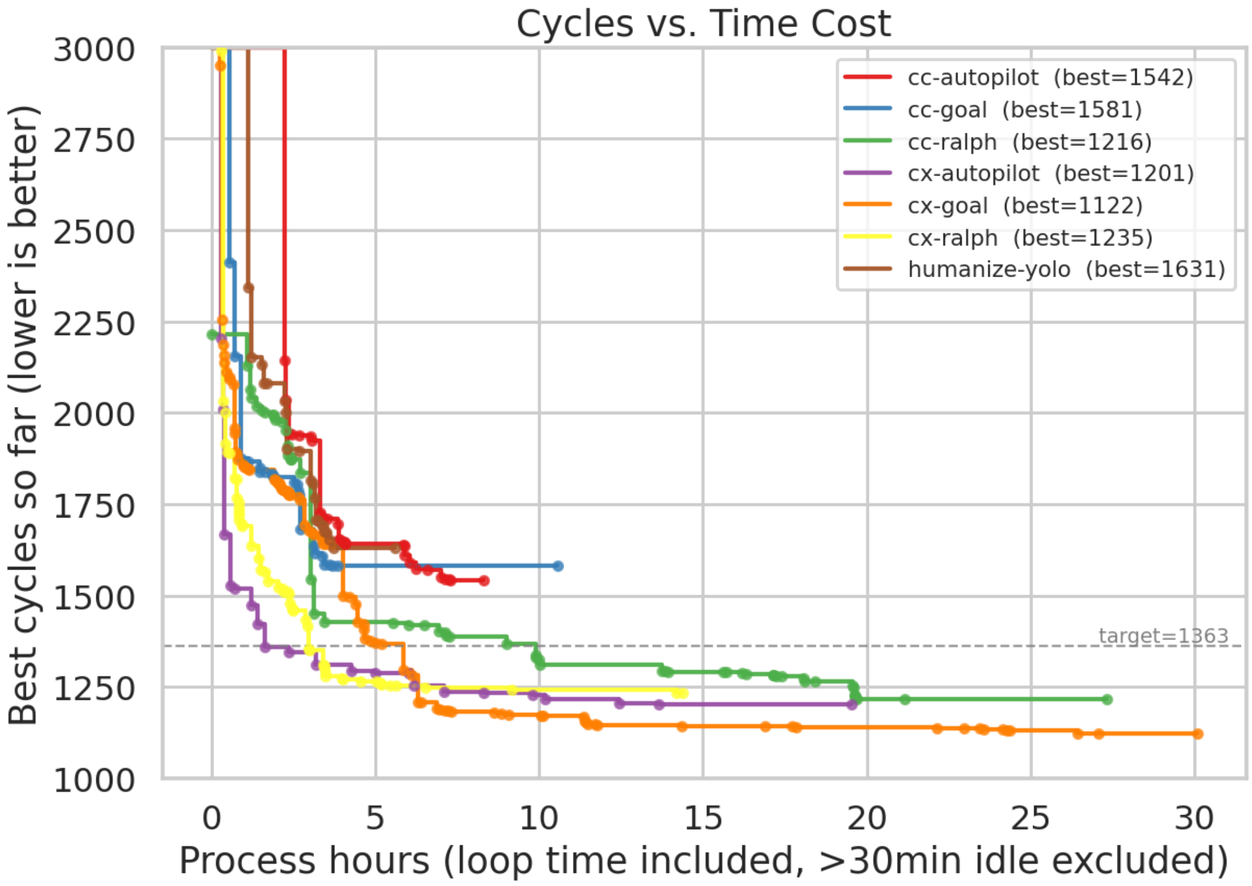
https://vliw-challenge.fly.dev/

NVIDIA

“Flow” Matters in Agent Productivity

Optimization progress: 7 methods on the perf_takehome kernel





NVIDIA

HOA: Humanize Olympic Agents

Modern SOTA LLMs are powerful, but to unleash its full power, we need Model X Tool cooperation.

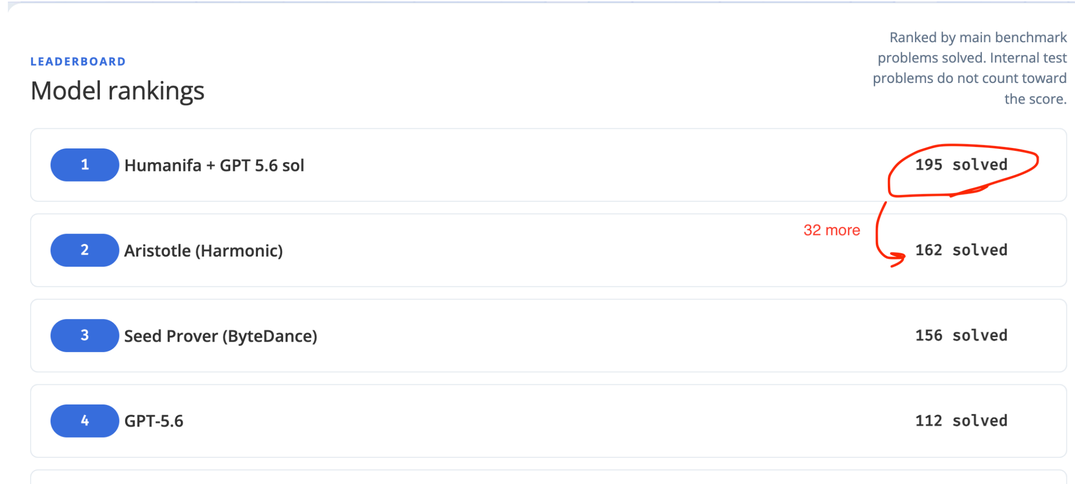
```
Vibe Code / Vibe Research =
    Model (opus/gpt/deepseek/kimi) X
    Tool  (claudecode/codex/opencode) X
    Flow  (humanize/ship/autoresearch/superpower)
```

1. **Humanize2 Flame Chase:** Sol and Fable work iteratively

2. **{Math,Physics,Chem}LIB**: A curated reference from text books

3. **FormalEval.skill**: using lean and comparator for solution evaluation

- Perfect score on IOI 2026 / ICHO2026 / IPHO2026 / IMO2026
  - All one pass, yolo mode, no human interaction
  - W/ humanize harness,
    - IMO2026, Codex + /goal: **4/6**, Codex + Humanize: **6/6**
    - Kimi-K3 can also solve (1.2x tokens on IMO and 8.3x token on IPHO)
- 195 problems and #1 ranking on Lean-Eval.
  - 33 problems ahead to 2nd player (harmonic, $2B value)
  - Beat SeedProver by a large margin (39 problems ahead)
  - Compared to pure GPT-5.6 + /goal, we solve 83 more problems!
- 670 / 672 excellent score on PutnamBench.
  - A curated problem set from 1963-2023.



NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

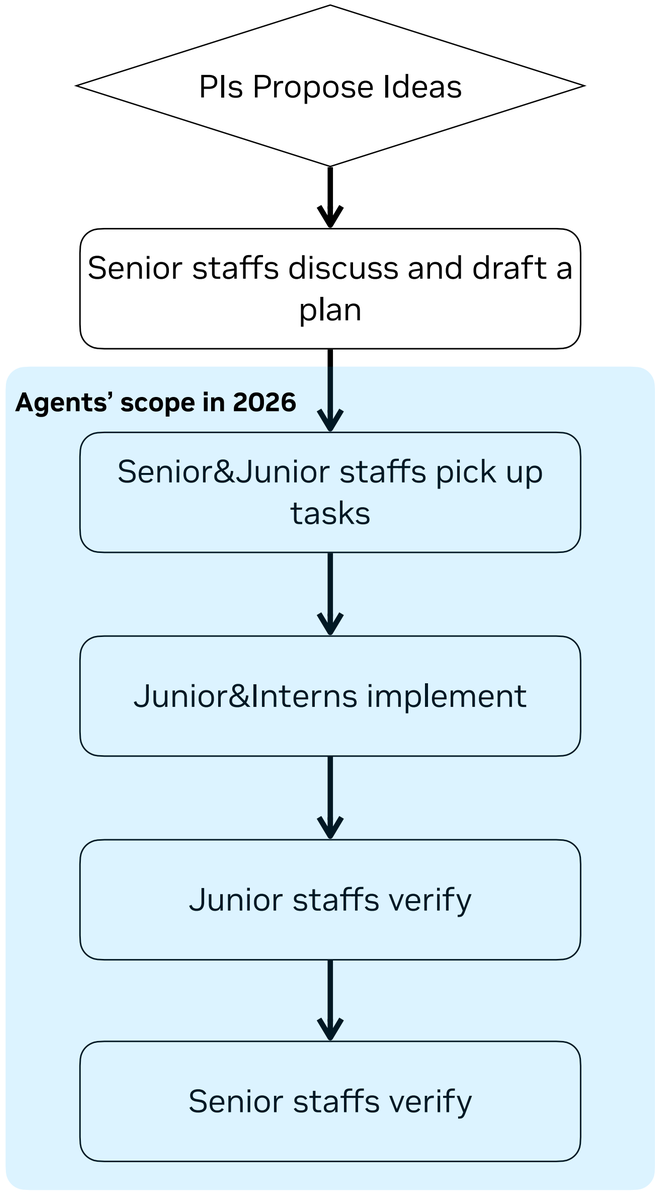
Take-aways

- Vibe Code / Vibe Research =
  Model (opus/gpt/deepseek/kimi) +
  Tool (claudecode/codex/opencode) +
  Flow (humanize/autoresearch/ralph-loop)

- If you miss any piece of the components, your vibe coding experience would suffer greatly!

- When all people use the same model (GPT/Opus) and same tool (Codex/Claude Code), Flow is key to improve the performance of agentic systems.

- Trust your agent as trust your lab-mates
  - Give him the full-access and full-approval
  - Talk to him w/ nature languages as much as possible, rather than interactive coding



O(#tokens) = O(#productivity)!

Humanize — past, present, and future

From Automated Idea Factory to Realization

Ligeng Zhu

NVIDIA Research

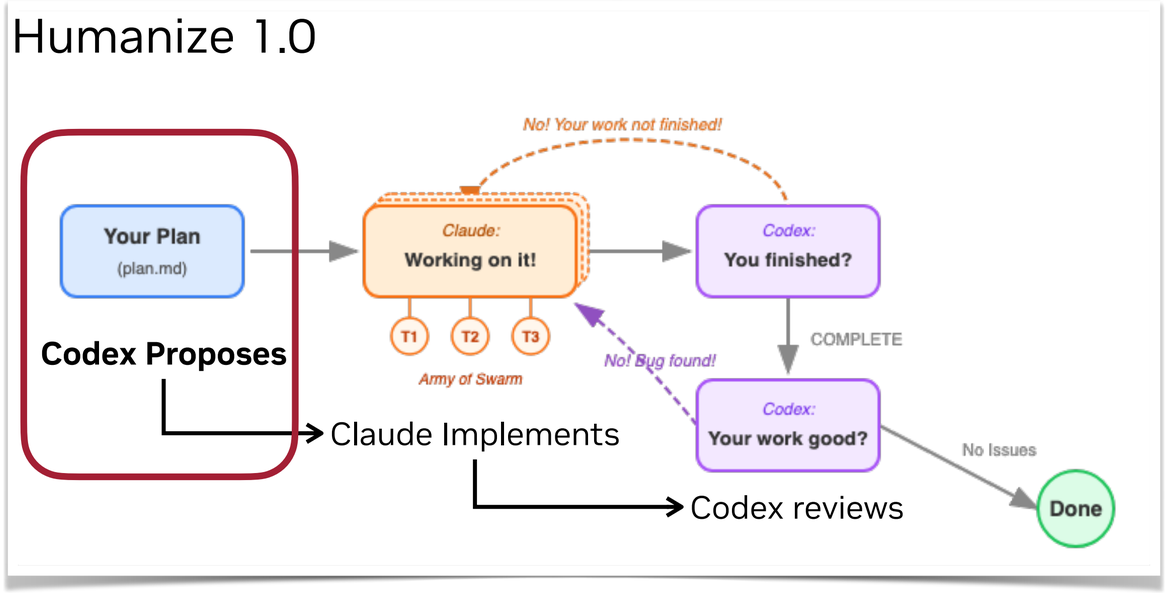
Humanize — Design Initiatives

- **Belief**:
  - LLMs are much more knowledgeable than human (>>30T token pre-train + post-train)
  - LLMs are much more skillful than human (IMO, IPhO, IOI, ACM, Putnam, Erdos math)
  - LLMs remembers more tricks / tips than human (1M ctx >> humans' 20~30k)
  - 7x24 agents would eventually outperform human (never tired / rest)

- **OK, but why my agents fails?**

Humanize — Design Initiatives

- **Agent failure comes from**
  - Mis-aligned initiative — Hugh ambiguity in natural language



- A heave and deep think to ensure the agents and human are on the same page.

Humanize — Design Initiatives

- **Agent failure comes from**
  - Mis-aligned initiative — Hugh ambiguity in natural language

Fast Implementation X

Weak Alignment X

Zero Judgement

= Fast Wrong Garbage Work (**boom**!)

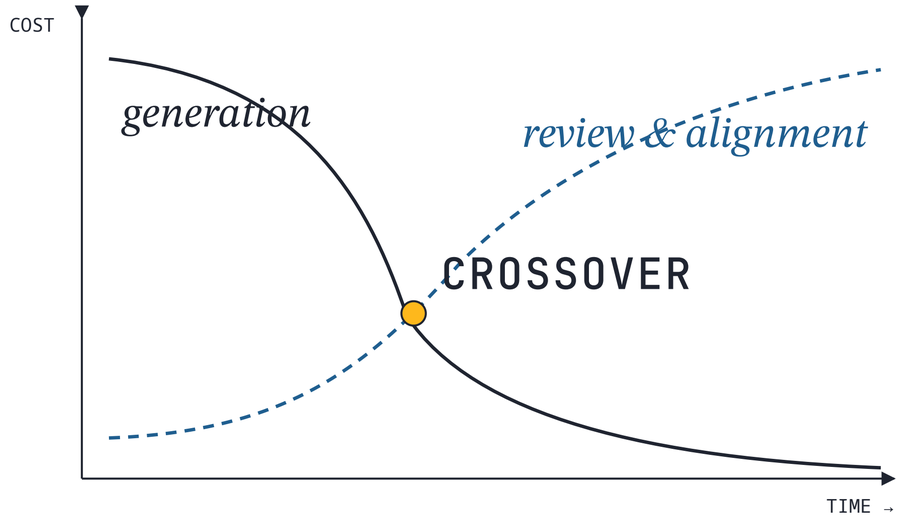
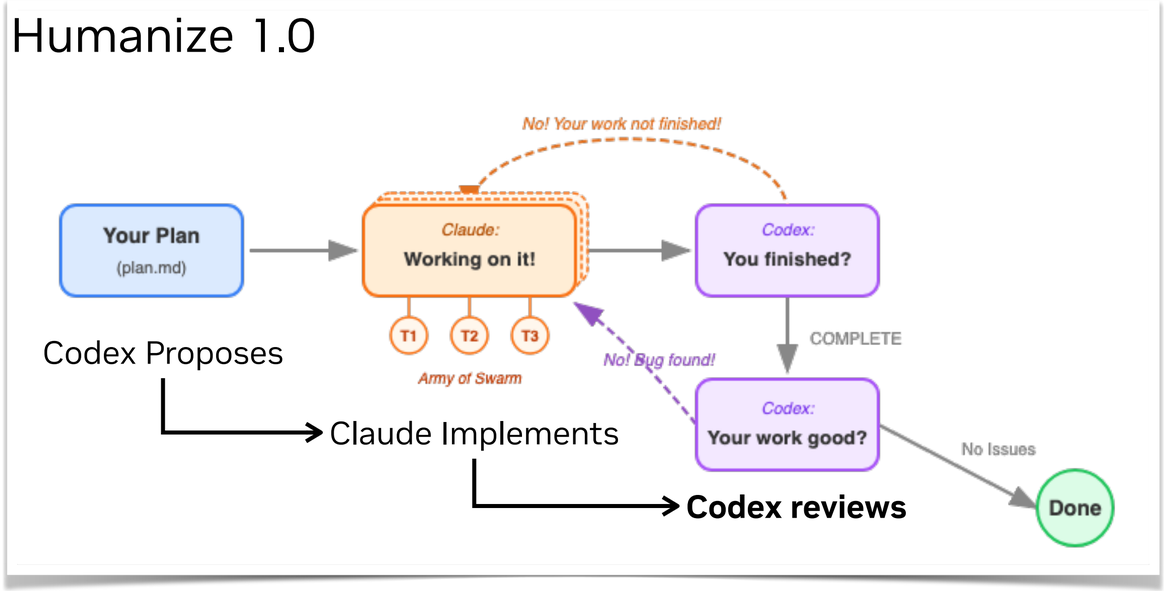


FIG. 1 — BOTTLENECK SHIFT IN AGENTIC CODING WORKFLOWS.

- Further, Humanize 1.0 **proposes quiz** to check human really understands the initiative
  - Check and ensure the agents are aligned with human.

Humanize — Design Initiatives

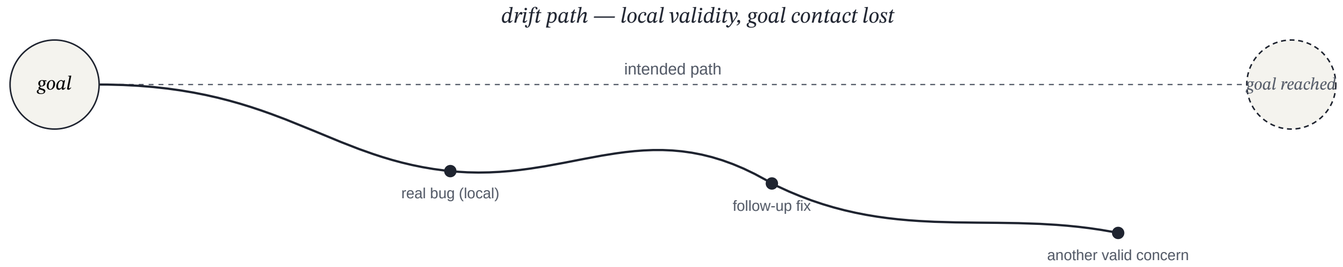
- **Agent failure comes from**
  - Mis-aligned initiative — Hugh ambiguity in natural language
  - Smart = tricky —— if an agent cannot find a way to hack “reward”, it is not smart enough.



- Must been peer-reviewed to be merged to proceed the next action bullet!
- Peer-Review is crucial to avoid the goal drifting.

Humanize — Design Initiatives

- **With goal alignment, and progressive task peer-review**, humanize 1.0 can avoid goal drifting and let agents run for days ~ weeks.



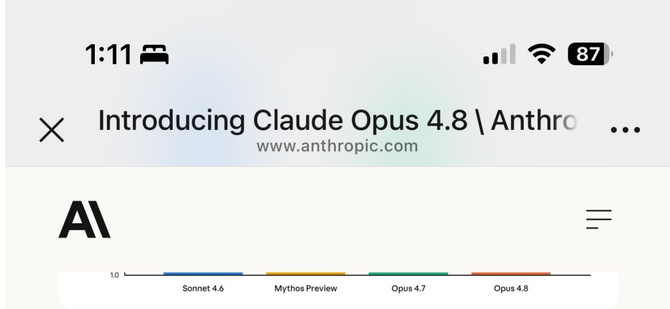
Design response: the progress verdict asks whether the loop still serves the plan, not whether the latest diff is correct. Side issues get a lane, not the steering wheel.

- Humanize 1.0 is **highly motivated by the way that human collaborates** on the Github
  - When you have important feature to merge
    - Draft a proposal (humanize draft.md —> plan.md)
    - Discuss with maintainers and polish (humanize refine plan.md)
    - Implement each sub-feature, then peer reviewed by others (humanize Claude implements — codex reviews)

NVIDIA

Humanize — Design Initiatives

- **From a company / organization angel:**
  - Model ~= Clever and top new grads
  - Tool ~= The Cluster / Office Room / Chair / Company Infra
  - Flow ~= Management / Org Design / Company Culture / Proper KPI
- The idea of Humanize's flow starts in Jan 2026, now partially introduced by Anthropic official in May 2026.
- ***Flow would be the most critical part*** of 2026～2027's research.



Also launching today

In addition to Claude Opus 4.8, we're making the following updates:

- **Dynamic workflows**. This new feature, available in research preview, allows Claude to take on even bigger tasks in Claude Code. Claude can plan the work and then run hundreds of parallel subagents in a single session (and with Opus 4.8, the agents can run for even longer). It then verifies its outputs before reporting back to the user. For example, Claude Code with Opus 4.8 can now carry out codebase-scale migrations across hundreds of thousands of lines of code from kickoff to merge, with the existing test suite as its bar. You can read more about dynamic workflows—available in Claude Code for Enterprise, Team, and Max plans—in **this post**.
- **Effort control in claude.ai and Cowork**. A new control alongside the model selector lets users choose how much

Humanize — Design Initiatives

- **From a company / organization angel:**
  - Model ~= Clever and top new grads
  - Tool ~= The Cluster / Office Room / Chair / Company Infra
  - Flow ~= Management / Org Design / Company Culture / Proper KPI

- I have a strong belief that Flow would be the most critical part of 2026～2027's research.

- Question: Different tasks require the same flow, or different flow?

- **<u>Humanize 2.0: an IR that allows user to write their own flows!</u>**

AISA — Agentic Instruction Set Architecture

Version 0.3, post-RISC reduction. AISA is the core intermediate representation of Humanize 2.0. A methodology program is a Development Flow Graph (DFG) whose nodes are AISA instructions and whose edges carry typed bundles of tokens through an ordered dataflow runtime (Kahn Process Networks dialect).

Token	Meaning	Discipline
!cb	Git-worktree handle (Codebase).	LINEAR — single consumer.
!ctx	LLM transcript (Context).	LINEAR — single consumer.
!p	Turn-sequence directive (Prompt).	BROADCAST — multi-reader.
!state	User-declared FSM variant	BROADCAST, scalar, never in bundle

Bundles

A bundle is a tuple shape over the three token kinds {p, cb, ctx}. AISA admits exactly seven non-empty bundles:

<p> <p,cb> <p,ctx> <p,cb,ctx>
<cb> <ctx> <cb,ctx>

```
state @RlcrState { @start Init, Build, Review, Finalize,
                   @end Done, @end Halted, @end Cancelled }
transitions for RlcrState {
    Init     -> Build,
    Build    -> { Review, Halted },
    Review   -> { Build, Finalize, Halted },
    Finalize -> { Done, Build, Halted },
}

agent @Builder  = Model(claude_opus_4_7) x Tool(claude_code) x
                  Permissions { Edit: Allow, Bash: Ask, SubAgent: Allow }
agent @Reviewer = Model(gpt_5_4)         x Tool(codex_cli)   x
                  Permissions { FileRead: Allow, Review: Allow }
agent @Id       = @hmz_stdlib::identity
agent @Final    = Model(gpt_5_4)         x Tool(codex_cli)   x
                  Permissions { FileRead: Allow, Review: Allow }

flow @rlcr over RlcrState (%init_cb: <cb>, %init_plan: <p>) -> <cb> {
    %ctx_init = aisa.branch(null) { agent = @hmz_stdlib::ctx_empty, M = 1 }

    // Loop-merge on three carriers
    %cb_iter  = aisa.converge(%init_cb,   %cb_back)
        { agent = @Id, M = 2, fire_in = ANY }
    %ctx_iter = aisa.converge(%ctx_init,  %ctx_back)
        { agent = @Id, M = 2, fire_in = ANY }
    %p_iter   = aisa.converge(%init_plan, %p_back)
        { agent = @Id, M = 2, fire_in = ANY }

    // Build phase: 1:1 on bundle <p,cb,ctx>
    %cb_b, %ctx_b, %p_b = aisa.step(%cb_iter, %ctx_iter, %p_iter)
        { agent = @Builder,
          trigger = "implement the plan, write round summary",
          transition = host { next = Review },
          verify = [@AC_BUILD] }

    // Review phase: 1:1 on bundle <p,cb,ctx>, agent-driven state
    %cb_r, %ctx_r, %p_r = aisa.step(%cb_b, %ctx_b, %p_b)
        { agent = @Reviewer,
          trigger = "grade the round summary; emit CONTINUE/COMPLETE/STOP",
          transition = agent {
              candidates    = [Build, Finalize, Halted],
              parse         = sentinel {
                  CONTINUE -> Build,
                  COMPLETE -> Finalize,
                  STOP     -> Halted,
              },
              on_ambiguous  = stay,
              max_ambiguity = 2,
          } }

    // State-driven switch on cb (and on ctx, p in parallel)
    %cb_back, %cb_to_final, %cb_halt = aisa.branch(%cb_r)
        { agent = @Id, M = 3, fire_out = ANY,
          select_by_state = [Build, Finalize, Halted] }
    %ctx_back, %ctx_to_final, %ctx_halt = aisa.branch(%ctx_r)
        { agent = @Id, M = 3, fire_out = ANY,
          select_by_state = [Build, Finalize, Halted] }
    %p_back, %p_to_final, %p_halt = aisa.branch(%p_r)
        { agent = @Id, M = 3, fire_out = ANY,
          select_by_state = [Build, Finalize, Halted] }

    // Finalize phase: code review by codex
    %cb_final, %ctx_final, %p_final = aisa.step(%cb_to_final, %ctx_to_fina
%p_to_final)
        { agent = @Final,
          trigger = "perform full code review; emit Done / Build / Halted"
          transition = agent {
              candidates = [Done, Build, Halted],
              parse      = predicate(@hmz_stdlib::codex_review_predicate),
              on_ambiguous = err(InvalidOutput),
          } }

    return %cb_final
}
```

Humanize 1.0 RLCR Example

From ~700 Loc bash to ~90 Lines

NVIDIA

AISA — Agentic Instruction Set Architecture

Version 0.3, post-RISC reduction. AISA is the core intermediate representation of Humanize 2.0. A methodology program is a Development Flow Graph (DFG) whose nodes are AISA instructions and whose edges carry typed bundles of tokens through an ordered dataflow runtime (Kahn Process Networks dialect).

Token	Meaning	Discipline
!cb	Git-worktree handle (Codebase).	LINEAR — single consumer.
!ctx	LLM transcript (Context).	LINEAR — single consumer.
!p	Turn-sequence directive (Prompt).	BROADCAST — multi-reader.
!state	User-declared FSM variant	BROADCAST, scalar, never in bundle

Bundles

A bundle is a tuple shape over the three token kinds {p, cb, ctx}. AISA admits exactly seven non-empty bundles:

<p> <p,cb> <p,ctx> <p,cb,ctx>
<cb> <ctx> <cb,ctx>

Humanize 2.0 Preview
5-way Parallel Build-Review

```
flow @parallel_rlcr (%init_cb: <cb>, %mega_plan: <p>) -> <cb> {
    %ctx_empty = aisa.branch(null) { agent = @hmz_stdlib::ctx_empty, M = 1 }

    // Plan split: 1:5 on <p>
    %p1, %p2, %p3, %p4, %p5 = aisa.branch(%mega_plan)
        { agent = @plan_splitter, M = 5, fire_out = ALL }

    // Codebase fork: 1:5 on <cb>
    %cb1, %cb2, %cb3, %cb4, %cb5 = aisa.branch(%init_cb)
        { agent = @hmz_stdlib::worktree_fork, M = 5, fire_out = ALL }

    // Five parallel RLCR lanes, each a nested flow call
    %r1 = aisa.step(%cb1, %ctx_empty, %p1) { agent = @rlcr }
    %r2 = aisa.step(%cb2, %ctx_empty, %p2) { agent = @rlcr }
    %r3 = aisa.step(%cb3, %ctx_empty, %p3) { agent = @rlcr }
    %r4 = aisa.step(%cb4, %ctx_empty, %p4) { agent = @rlcr }
    %r5 = aisa.step(%cb5, %ctx_empty, %p5) { agent = @rlcr }

    // Merge: 5:1 on <cb>
    %final = aisa.converge(%r1, %r2, %r3, %r4, %r5)
        { agent = @merge_coordinator, M = 5, fire_in = ALL,
          trigger = "consolidate 5 candidate branches; keep the strongest" }

    return %final
}
```

Humanize 2.0 Preview
100-Idea Stream + Parallel Evaluate

```
flow @idea_hunt (%cb_seed: <cb>, %seed_prompt: <p>) -> <cb> {
    %ctx_empty = aisa.branch(null) { agent = @hmz_stdlib::ctx_empty, M =

    // Brainstorm 100 ideas as a 1:100 branch; M is static
    %i1, ..., %i100 = aisa.branch(%seed_prompt)
        { agent = @brainstormer, M = 100, fire_out = ALL,
          trigger = "produce 100 distinct approaches" }

    // Fork the codebase 100 ways
    %cb1, ..., %cb100 = aisa.branch(%cb_seed)
        { agent = @hmz_stdlib::worktree_fork, M = 100, fire_out = ALL }

    // Evaluate each in parallel (compile-time map unroll)
    %e1 = aisa.step(%cb1, %ctx_empty, %i1) { agent = @build_review_profil
    ... ; %e100 = aisa.step(%cb100, %ctx_empty, %i100) { agent =
@build_review_profile }

    // Rank and pick best: 100:1 on <cb>
    %best = aisa.converge(%e1, ..., %e100)
        { agent = @ranker, M = 100, fire_in = ALL,
          trigger = "score each candidate, keep the top-scoring branch" }

    return %best
}
```

NVIDIA

Humanize — Design Initiatives

- **From a company / organization angel:**
  - Model ~= Clever and top new grads
  - Tool ~= The Cluster / Office Room / Chair / Company Infra
  - Flow ~= Management / Org Design / Company Culture / Proper KPI

- **From an IR / Programming Language / Computer Architecture view**
  - Model ~= CPU
  - Tool ~= OS
  - Flow ~= Program (.py .cpp)
    - The new programming language that undergrad students will learn 3~5 years later
    - And, humanize 2.0 wants to be the language

Humanize — Design Initiatives

- **Humanize 1.0**: A flow that works.
  - A single hard-coded RLCR loop. Builder + Codex reviewer, ralph-loop semantics, severity tags. One way to do one thing — and it does it well.

- **Humanize 2.0:** An IR that builds flows; enabling flows to call flows.
  - Flow primitives in HTML, typed artifacts & boards, a trigger-driven executor, tool-agnostic agents, a local hub for observation & control.

- **Humanize 3.0:** A platform for flow IRs; targeting flows to generate flows.
  - Once flow is HTML, agents can author flows. The system bootstraps its own coordination logic — h2 is the substrate that makes h3 possible.

Humanize — Design Initiatives

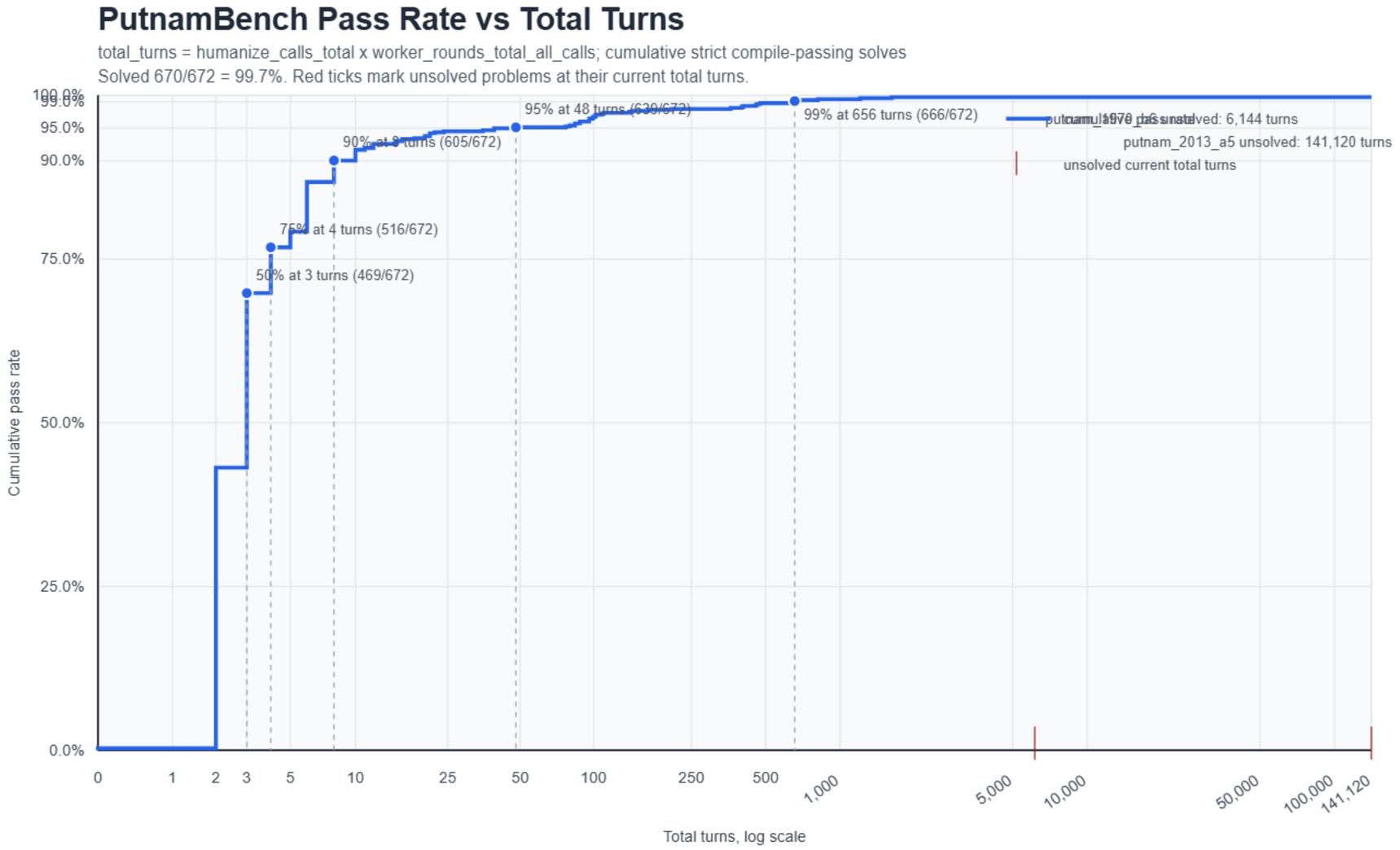
- **Humanize 1.0**: A flow that works.
  - A single hard-coded RLCR loop. Builder + Codex reviewer, ralph-loop semantics, severity tags. One way to do one thing — and it does it well.

- **Humanize 2.0:** An IR that builds flows; enabling flows to call flows.
  - Flow primitives in HTML, typed artifacts & boards, a trigger-driven executor, tool-agnostic agents, a local hub for observation & control.

- **Humanize 3.0:** A platform for flow IRs; targeting flows to generate flows.
  - Once flow is HTML, agents can author flows. The system bootstraps its own coordination logic — h2 is the substrate that makes h3 possible.

Back to 2023, people found LLMs are powerful, but needs prompting to guide. —> Prompt Engineering
Move to 2025, people realize the best prompts for LLMs are written by itself ——> Reasoning
Mid in 2026, people found agents are capable, but needs flows to manage ——> Humanize 1.0
(?) sometime soon, people realize the best flows are written by agents itself ——> Humanize 3.0

Humanize — The Showcase of Power of the “Flow”



Putnam-mathbench 99.7% (w/ Humanize 2.0 designed flow)

Previous best SOTA from princeton’s Goedel-Architect 88.9% (just reread June 04, 2026)

H1 · PAST

v1

A flow that works.

A single hard-coded RLCR loop. Builder + Codex reviewer, ralph-loop semantics, severity tags. One way to do one thing — and it does it well.

- Plugin commands
- Bash + worktree files
- Hard-coded loop shape

V 1.17.0 · SHIPPED

H2 · PRESENT

v2

A platform/language humans build flows on.

Flow primitives in HTML, typed artifacts & boards, a trigger-driven executor, tool-agnostic agents, a local hub for observation & control.

- 12 vertex primitives
- Triggered dataflow
- MCP hub + GUI

BRANCH H2-DEV · IN FLIGHT

H3 · FUTURE

v3

Agents create agentic flow.

Once flow is HTML, agents can author flows. The system bootstraps its own coordination logic — h2 is the substrate that makes h3 possible.

- Agents author cartridges
- Self-deploying flows
- Bootstrapped coordination

UNSCHEDULED · SCOPED

NVIDIA

JetAutoResearch

A token-efficient harness for agent-driven research

Ligeng Zhu

NVIDIA Research

JetAutoResearch - Method in One Slide

AutoResearch is cool, can we reduce the cost to leverage more people to use the harness?

Karparthy's AutoResearch is a mini agent framework that makes an LLM behave like an overnight research intern, but the cost soon grows rapidly (~$5k token cost per week if running on a 8-GPU node) and soon you will see the page below 😭

Hello,

Your API access has been paused in the Claude Code workspace because you exceeded your $10000.00 monthly spending limit. API access will resume at midnight UTC on May 1, 2026.

Warmly,
The Anthropic Team

JetAutoResearch - Method in One Slide

AutoResearch is cool, can we reduce the cost to leverage more people to use the harness?

 # program.md
LOOP FOREVER:
1. Look at the git state: the current branch/commit we're on
2. **Tune `train.py` with an experimental idea by directly hacking the code.**
3. git commit
4. Run the experiment: `uv run train.py > run.log 2>&1` (redirect everything — do NOT use tee or let output flood your context)
5. Read out the results: `grep "^val_bpb:\|^peak_vram_mb:" run.log`
6. If the grep output is empty, the run crashed. Run `tail -n 50 run.log` to read the Python stack trace and attempt a fix. If you can't get things to work after more than a few attempts, give up.
7. Record the results in the tsv (NOTE: do not commit the results.tsv file, leave it untracked by git)
8. If val_*bpb improved (lower), you "advance" the branch, keeping the git commit*
*9. If val_*bpb is equal or worse, you git reset back to where you started

Two types of workloads:

1. the *high-value reasoning for creative tasks* that benefit from the model's intelligence, such as proposing code changes, diagnosing crashes, and suggesting new directions.
2. the ***low-value orchestration for the workflow*** **(65% tokens)** that manages state transitions, retries, and branch advancement or rollback.

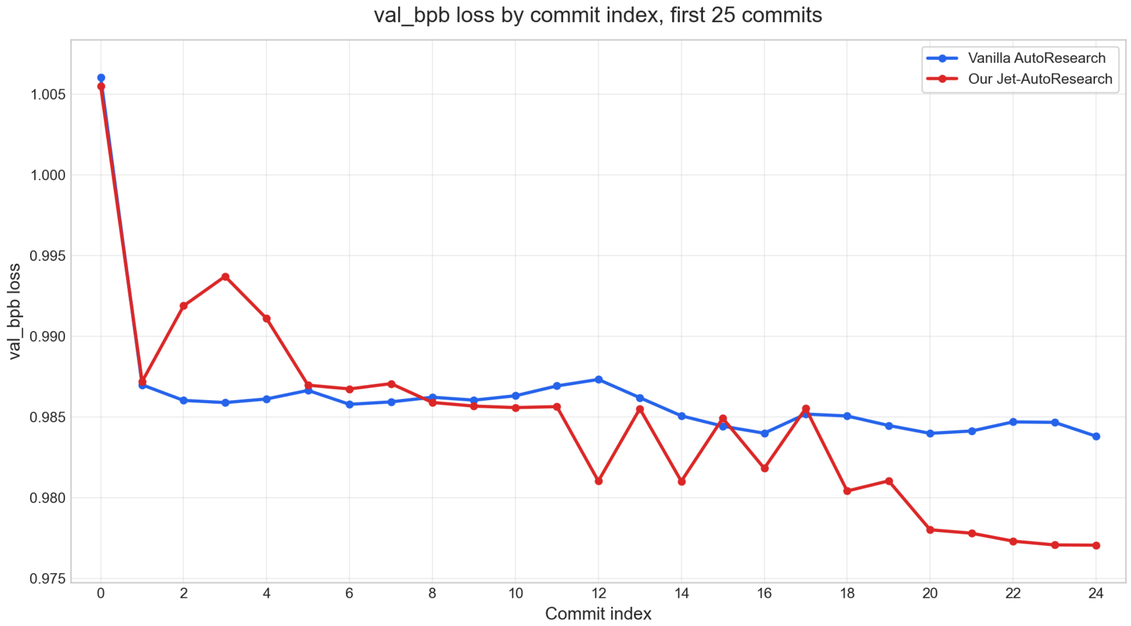
Main Idea:

"Compile" the low-value orchestration from LLM execution to python/ bash scripts.

JetAutoResearch - Results

Same Results, but much less tokens.

The same optimization convergence speed, but much less cost (on GPT5.4 xhigh fast)



Easy to use as an ahead-of-time compiler for harness

```
# vanilla autoresesarch
codex  “launch the @program.md”


# jet-autoresearch
codex “$jet-compile-plan program.md program.agentify.py"
codex “python program.agentify.py”
```

1. Average run cost drops from about `$17.1` to `$7.6` (`**54%` lower**).
2. Average decode tokens drop from `24,251` to `4,593` (`**71%` lower**).

“

Thanks, questions?

”